Rupa

Plan, guide, and tutorial for a renderer layer for Claude Code

Vajra Sarpa · October 15, 2026

CONTENTS

About this document

Part 1: Plan

The problem
The idea
Goals and non-goals
Architecture
Key decisions
Milestones
Success metrics
Risks

Part 2: Guide

The contract
Input format
Themes
Verification
Plugin anatomy
Token discipline
Measuring the savings
Well-Architected lens

Part 3: Tutorial

Before you start
Step 1: Scaffold the project
Step 2: The document model
Step 3: The renderer and themes
Step 4: The CLI contract
Step 5: Install it as a command
Step 6: Continuous integration
Step 7: The Claude Code plugin
Step 8: Verification
Step 9: Measure
Step 10: What to build next

Turning the build into a showcase

Appendix: Verification record

Pass 1: the first delivered version
Pass 2: after adding verification
Final pass

About this document

Rupa (Sanskrit *rūpa*, "form") is a small command-line renderer. Claude writes content as Markdown; Rupa owns the form, turning that Markdown into a styled HTML page or PDF. A thin Claude Code plugin teaches Claude to use it.

The document has three parts plus a short note on presenting the build:

- **Part 1, Plan:** what to build, why, the decisions, milestones and risks.
- **Part 2, Guide:** the reference for how Rupa works once it exists.
- **Part 3, Tutorial:** build it step by step, from an empty folder to a tested CLI, a validated plugin, a verification loop and a benchmark.

Everything in Part 3 was built and run before this was written: 40 tests pass, `claude plugin validate` passes, and Claude Code loads the skill and uses it to render and verify a PDF on its own. This document was rendered by that prototype from one Markdown file, then checked with `rupa verify` and a visual review of its sample pages. The appendix records what that review found.

Part 1: Plan

The problem

When Claude Code produces an HTML page or PDF, it writes the markup itself: every tag, every CSS rule, or a script that builds the PDF. Four costs follow.

- **Turns.** Finding a PDF tool, installing it, rendering, and debugging the result each take a turn, and every turn re-reads the whole conversation from cache. In the one run measured during the build, cache reads were about 99% of the tokens processed, so turn count is what drives cost.
- **Output tokens.** Markup and styling are a large share of what Claude writes. They are the slowest tokens, but a small share of the total.
- **Revision cost.** A one-line content fix often regenerates the whole file, styling included.
- **Inconsistency.** Each document is styled from scratch, so two reports never look quite the same, and render bugs are fixed in one file at a time.

The idea

Split content from form. Claude emits compact Markdown with a few frontmatter keys. Rupa, a deterministic and tested program, applies the theme and produces the file. Rupa prints one status line, so Claude doesn't read the rendered output back by accident.

Then check the result where it matters, because people read these documents and people like pretty things. Rupa runs code checks on the PDF and prepares a small set of page images, each next to the matching page of the theme's reference. Claude looks at those deliberately. Tokens are saved on generation and spent on review.

Without Rupa	With Rupa
Claude writes HTML, CSS, PDF code	Claude writes Markdown
Styling re-emitted on every revision	Styling lives in Rupa themes
Claude re-reads the whole output to check it, or doesn't check	Code checks every page; Claude looks at a few sampled pages next to a reference
Every document styled ad hoc	One theme, applied the same way every time

Goals and non-goals

Goals

1. Markdown plus frontmatter in, HTML or PDF out, from stdin or a file.
2. stdout carries one status line (or one JSON object), never the document.
3. Clear exit codes so Claude can fix input without guessing.
4. PDF body text is solid black (#000000, full opacity).
5. A Claude Code plugin under about 30 lines of instructions, with no always-on MCP tool definitions.
6. Every PDF verified twice: code checks for what can be measured, and a visual review against the theme's reference pages for what can't.
7. A repeatable benchmark that measures turns and cost per finished document instead of assuming them.

Non-goals for v1

- A layout language. If Markdown cannot express it, it waits for a later version.
- DOCX or PPTX output. They can be added as new targets behind the same contract.
- Remote assets. Images must be relative paths or `data:image` URIs, so neither HTML nor PDF fetches from the network.

Architecture

The pipeline is linear and each stage has one job.

```
Markdown + frontmatter   (stdin or a file)
        │
        ▼
parse()                  document.py   → Document(meta, body_html, headings)
        │
        ▼
to_html()                render.py + themes/*.css
        │
        ├──► .html file
        └──► to_pdf()    WeasyPrint    → .pdf file
                                              │
                                              ▼
verify()                 verify.py     → checks + sheets + verify.json
        │
        ▼
```

```
Claude reads the sheets, judges them against the rubric, fixes the Markdown

stdout: one status line          exit code: 0 / 2 / 3 / 4 / 5
```

Module	Responsibility	Knows about styling?
`document.py`	Parse frontmatter, Markdown, headings	No
`render.py`	HTML template, theme CSS, PDF	Yes, and only here
`themes/*.css`	All visual decisions	Yes
`verify.py`	PDF checks, page types, review sheets	Reads the result, never styles it
`references/`	Reference pages for each theme and page type	They are the theme's picture of itself
`cli.py`	Arguments, I/O, status line, exit codes	No
`plugin/`	Teach Claude the contract	No

Key decisions

Decision	Choice	Why	Rejected
Interface	CLI over stdin/stdout	Works in Claude Code, CI, scripts and any harness; zero context cost until called	MCP server: tool schemas sit in context every turn
Input format	Markdown + YAML frontmatter	Claude writes it fluently; few tokens; no schema to teach	Custom JSON schema: more prompt overhead and more retries
PDF engine	WeasyPrint	Pure Python install, real CSS paged media (page numbers, running titles)	Headless Chrome: heavy dependency; LaTeX: a second styling system
Markdown parser	markdown-it-py	CommonMark compliant, plugin system, token stream for headings	Python-Markdown: weaker spec compliance
CLI library	argparse	No dependency, enough for six flags	Typer/Click: nice, not needed yet
Raw HTML in input	Escaped	Keeps Claude from smuggling styling back in; safer output	Pass-through

Decision	Choice	Why	Rejected
Plugin form	One skill + one hook	Skill body loads only when triggered	Many slash commands
Verification	Code checks, then vision on sampled pages beside reference pages	Code is exact and free; vision judges what code can't. Sampling by page type keeps the cost tied to variety, not length	Vision on every page (~85k tokens for 31 pages); pixel diffs (content always differs from the reference)
Who looks at the images	Claude, through the skill	Rupa stays offline with no API key; Claude is already a vision model	Rupa calling a model API itself

Milestones

Estimates assume evenings and weekends.

#	Milestone	Done when	Estimate
M0	Scaffold	`uv sync` and an empty test run pass	1 hour
M1	Parser	Document model and 8 parser tests pass	2–3 hours
M2	Renderer	HTML and PDF from a sample; PDF text forced black	3–4 hours
M3	CLI contract	Status line, `--json`, exit codes, CLI tests pass	2 hours
M4	Plugin	`claude plugin validate` passes; skill loads with `--plugin-dir`	1–2 hours
M5	Verify	`rupa verify` checks and sheets; reference pages built; skill runs the review loop	4–6 hours
M6	Measure	Pilot of one run per arm, then three runs per arm; medians recorded	2 hours plus about $2–6 of API use
M7	Ship	CI green on GitHub Actions; `uv tool install` from the repo works	1–2 hours

Success metrics

- **Primary:** median turns and median cost (`total_cost_usd`) per finished document, Rupa against baseline. Turns come first because each one re-reads the whole context, so fewer turns is where most of the saving should come from. Report the numbers you get, whichever way they go.
- **Secondary:** median cache reads and output tokens, and the success rate (runs that produced the PDF).
- **Verification overhead:** the difference between the `rupa` and `rupa-verify` arms, which is what the visual review costs per document.
- **Quality:** all tests green; the same input renders the same output every time; every delivered PDF passes `rupa verify` and a visual review with no open failures.

Risks

Risk	Likelihood	Impact	Mitigation
Savings smaller than expected	Medium	Medium	Pilot in M6 before investing in themes; judge on turns and cost, not output tokens alone
Vision verdicts vary between runs	Medium	Medium	Fixed rubric, one structured verdict per sheet, code checks for anything measurable, two fix rounds at most
Code checks misread a page (page type, fill)	Medium	Low	Unit tests on synthetic pages; flagged pages always go to the visual review, so a wrong flag costs one sheet
Claude ignores the skill and hand-writes HTML	Medium	High	Short, direct skill description; check with the benchmark transcripts
WeasyPrint system libraries missing on a machine	Medium	Low	Exit code 4 with a clear message; install notes in the tutorial; CI installs them
Markdown can't express a layout the user wants	High over time	Medium	Add themes and small, explicit extensions; never accept raw HTML
Library API changes (this happened during the build: WeasyPrint 70 replaced its URL fetcher API)	Medium	Low	Pin minimum versions; tests cover the behavior, not the library call

Part 2: Guide

The contract

Rupa has one contract, and the plugin, tests and benchmark all depend on it.

Channel	Carries
stdin or file argument	Markdown with optional YAML frontmatter
`-o PATH`	The rendered file. Format comes from the extension, or `--to html` / `--to pdf`. With `--to`, an existing file that isn't `.html` or `.pdf` is replaced only with `--force`
stdout	One status line, or one JSON object with `--json`. Never the document
stderr	One error line on failure
exit code	`0` ok, `2` usage, `3` bad input, `4` render failure, `5` verify checks failed or a PDF still over its page budget

Two subcommands follow the same rules:

Command	Does	Prints
`rupa verify OUT.pdf [--wants F]`	Runs the checks, writes sheets and `verify.json`	One line: checks passed, failures, sheet count
`rupa refs [--theme T]`	Rebuilds a theme's reference pages from the bundled sample	One line: which sample page became each reference

A Markdown file named `verify` or `refs` is read as the subcommand; write `./verify` to render it.

Example:

```
$ rupa notes.md -o out/notes.pdf
rupa: wrote out/notes.pdf (pdf, 3 pages, 48,211 bytes)

$ rupa notes.md -o out/notes.pdf --json
{"ok": true, "path": "out/notes.pdf", "format": "pdf", "pages": 3, "bytes": 48211}
```

Fonts

PDFs embed Inter and JetBrains Mono from `src/rupa/themes/fonts/` (both SIL Open Font License 1.1, licences alongside). Before that, a PDF used whatever the machine had: on a machine without Inter the theme fell back to Liberation Sans, the reference pages no longer matched the renders, and the fit numbers drifted. The image guard lets the renderer read that one folder besides the document's own.

Page budgets

`pages: N` in the frontmatter (or `--pages N`) gives a PDF a page budget. If the first render is over, Rupa tightens the form, never the content, one step at a time: it drops the contents list, then sets compact type, then tight type. Margins and heading sizes stay fixed, because `verify` measures against them. The status line names the steps used:

```
rupa: wrote out/report.pdf (pdf, 2 pages, fit to 2 with no contents + compact type, 41,322
bytes)
```

If even tight type is over, Rupa still writes the file. It exits `5` and says how many words sit past the budget, so Claude cuts that much in one edit instead of measuring pages itself:

```
rupa: wrote out/report.pdf (pdf, 3 pages, 44,870 bytes); over the 2-page budget at the
tightest type: cut about 140 words and render again
```

Input format

Frontmatter is optional. Only these keys are accepted; anything else is exit code 3, so typos fail loudly instead of being ignored.

Key	Type	Effect
`title`	string	Cover title and PDF running footer. Defaults to the first heading
`subtitle`	string	Line under the title
`author`	string	Byline
`date`	string	Byline, after the author
`theme`	string	Theme name. Defaults to `default`; `--theme` overrides it

Key	Type	Effect
`toc`	bool	Table of contents from `#` and `##` headings. Defaults to true

Body conventions:

- `#` is a part. In PDF each part starts a new page, so the skill keeps parts for long documents (about 8 pages or more); in a short one they leave half-empty pages.
- `##` is a section, `###` a subsection. Sections get anchors automatically.
- Fenced code blocks take a language tag for highlighting.
- Tables use GitHub pipe syntax. `~~strikethrough~~`, task lists (`- [ ]`, `- [x]`) and footnotes (`[^1]`) work too; in a PDF the footnotes are collected at the end.
- Raw HTML is escaped and shown as text.
- Images must be relative paths next to the Markdown file, or `data:image` URIs. A URL or absolute path is bad input (exit code 3); a missing file, a hidden one, one outside the Markdown file's folder, or one that doesn't decode as a PNG, JPEG, GIF, WebP or SVG image fails the render (exit code 4). HTML output embeds each image as a data URI, so the file can be sent on its own.

Themes

A theme is one CSS file in `src/rupa/themes/`. The PDF target appends `print.css` on top of it, which sets the page size, margins, footer, page breaks, and forces all text to solid black:

```
body, p, li, td, th, h1, h2, h3, h4, blockquote, code, pre,
.subtitle, .byline, .toc a, a {
  color: #000000 !important;
  opacity: 1 !important;
}
.hl * { color: #000000 !important; font-style: normal !important; }
```

The cyan accent survives only on rules and borders, never on text. Syntax highlighting is turned off in PDF for the same reason.

Code blocks use `white-space: pre-wrap` and nothing else. WeasyPrint treats `word-wrap: break-word` as permission to break anywhere, which split `solid` into `s` and `olid` in an earlier render of this document. The visual review caught it; the code checks couldn't.

To add a theme, copy `default.css` to `newname.css`, change the variables at the top, add the name to `THEMES` and its PDF text colours to `THEME_TEXT_COLORS` in `render.py`, and run `rupa refs --theme newname` to build its reference pages. The golden-render test then covers it with no new test code.

A theme can also carry a print layer, `newname.print.css`, appended after `print.css` for PDF only. Two themes use one: `cyan` and `cyan-dark`, built from the Cyan Signature system used for slide decks (deep navy, one cyan accent, Hanken Grotesk with JetBrains Mono labels, green-dot pill badges, rounded panels). They share `cyan.css` for screen through `THEME_BASE`. The `cyan` print layer turns the page white and every glyph black, keeping the type and cyan rules; `cyan-dark` keeps the navy page and prints in the deck's own ink, cyan and greys, which verify accepts because they're listed for it. Each render records its theme in the PDF's keywords, so `rupa verify` picks the right reference pages and colours without being told.

WeasyPrint shaped the print layers. It paints a page background from the text area down, so a glow at the top edge got a hard line; the dark PDF keeps the gradient and drops the glow. It has no `fit-content` and centres flex items by shrinking them, so pills are `display: table` and the PDF cover is a centred block. The sample's code page was 97% full in the default theme, so the pill above each part title pushed it onto a fifth page; the sample now has seven functions, not eight, to leave every theme room.

Verification

Rendering proves the file exists. Verification checks that it's right, in two tiers. **Tier 1: code checks.** `rupa verify` reads the PDF with pdfplumber and checks what can be measured exactly. Each check is free, repeatable and reports page numbers.

Check	Fails when
`text-color`	Any character isn't the wanted color (#000000 by default)
`page-size`	A page isn't Letter (or A4, if the wants file says so)
`max-pages`	The document is longer than the limit
`blank-page`	A page has no text
`near-empty`	A page is under 20% full (cover and last page exempt)
`sparse`	A page is under 40% full and the next page doesn't start a new part
`heading-at-bottom`	A heading is the last line on a page, or a heading and one line end it and the next page doesn't open with a heading

Check	Fails when
`overflow`	Text runs past the side margins

Tier 2: visual review. Code can't tell whether a page looks right. For that, `rupa verify` classifies each page as cover, prose, table or code. It then writes a sheet image for every page a code check flagged (with `sample: per-type`, one page of each type too), with the theme's reference page on the left and the rendered page on the right. Claude opens those sheets and judges each one against a fixed rubric: hierarchy, spacing, overflow, page balance and style.

The wants file

A `wants.yaml` holds what a finished PDF must satisfy. Every key is optional, unknown keys are an error, and `wants.example.yaml` in the repo spells out the defaults. A project that needs A4 and a 10-page limit writes two lines:

```
page_size: a4
max_pages: 10
```

Reference pages

A theme's reference pages are its picture of itself. `rupa refs` renders a bundled sample document with the theme and saves the most typical page of each type as a PNG inside the package. Rebuild them whenever the theme's CSS changes, and commit them with it. A golden-render test (`tests/test_golden.py`) holds you to that: for every theme it renders the sample, requires it to pass verify, rebuilds the reference pages in a temporary folder, and fails if any of them differs from the committed PNG in more than 0.5% of its pixels. The tolerance covers antialiasing differences between Pango builds; a moved margin fails it. A sheet without a reference still shows the page, labeled so.

What it costs

An image costs roughly its width times height divided by 750 in input tokens. A sheet at the default 90 DPI is about 1,530 × 1,020 pixels, or about 2,000 tokens.

How Claude reviews the 31-page version of this document	Images	Input tokens, approx.
Every page at full resolution	31	85,000
`rupa verify` : flagged pages plus one per type	5	10,000

How Claude reviews the 31-page version of this document	Images	Input tokens, approx.
Code checks only	0	under 100

The sampled review scales with how varied the document is, not how long it is. That is the right place to spend tokens: the reader sees the result.

The loop

The skill tells Claude to render, verify, open the sheets, and write one verdict per sheet. Claude fixes failures in the Markdown only: split a table that runs past the margin into two that repeat the first column, shorten a code line, move content around an awkward break. The `overflow` detail in `verify.json` says what ran over and by how much (`p1 table by 13pt`), so Claude knows which fix applies. Then it renders and verifies again, at most twice. A problem that can only be fixed in CSS is reported as a theme issue rather than patched, so one document's quirks don't leak into the theme.

Plugin anatomy

The plugin is deliberately small. Claude Code only needs to know that Rupa exists and how to call it.

```
plugin/
├── .claude-plugin/
│   └── plugin.json        # name, version, description, author
├── skills/
│   └── render/
│       └── SKILL.md       # the whole instruction set, ~40 lines
└── hooks/
    └── hooks.json         # SessionStart: warn if rupa is not on PATH
```

What each part costs in context:

Component	When it loads	Cost
Skill name and description	Every session	One line
Skill body	Only when the skill is triggered	About 40 lines
Review sheets	Only when Claude opens them during verification	About 2,000 tokens each

Component	When it loads	Cost
SessionStart hook	Session start	Prints nothing if `rupa` is installed
MCP server	Not used	Would be every turn

Claude Code namespaces plugin components under the plugin name, so the skill appears as `rupa:render`.

Two alternatives you may want later:

- `bin/` **directory.** Files in a plugin's `bin/` are put on the Bash tool's PATH while the plugin is enabled, so you could ship a `rupa` wrapper inside the plugin. The catch: claude.ai and Cowork don't install a plugin that has a top-level `bin/` directory, so this limits where the plugin can be used. Keep the CLI installed separately with `uv tool install` unless you only care about Claude Code.
- **MCP server.** Worth it only if Rupa grows state or a live preview. Until then it adds per-turn context for nothing.

Token discipline

The savings come from what Claude does not do. The skill enforces four rules:

1. Write Markdown, not markup.
2. Never put styling in the content. A different look is a theme change.
3. Read the status line, not the output file. Look at pages only through the review sheets, where looking is the point.
4. On failure, read the one stderr line and fix the input.

The fourth rule depends on Rupa's errors being specific. "unknown frontmatter keys: ['colour']" lets Claude fix the problem in one edit. A stack trace would cost a turn and a lot of tokens.

Measuring the savings

Where the cost goes

One run from the build, not a benchmark: asked for a one-page PDF summary of the six Well-Architected pillars, Claude Code with the plugin rendered, ran `rupa verify`, opened the sheet, and fixed the Markdown once to get the document to one page. No HTML or CSS was written by hand.

Measure	Value
Turns	16
Cache reads	636,178 tokens
Cache writes	57,643 tokens
Uncached input	28 tokens
Output	3,651 tokens
Total cost	$0.33

Output was about half a percent of the tokens processed. Every turn re-reads the conversation, so cost tracks turns. That changes what Rupa should be judged on: not mainly smaller output, but fewer turns. No hunting for a PDF tool, no render debugging, and one-line errors that are fixed in one edit.

Arms

`bench/bench.sh` runs the same task in four arms through Claude Code's print mode (`claude -p ... --output-format json`):

Arm	Setup	Answers
`baseline`	No plugin	What it costs today
`rupa`	Plugin with the skill's Verify section removed	What rendering through Rupa saves
`rupa-pertype`	Plugin whose skill runs `rupa verify --sample per-type`	What verify costs when every page type gets a sheet, even on a clean PDF
`rupa-verify`	Plugin as shipped (sheets only for flagged pages)	What the visual review adds back

The first benchmark called the flagged arm `rupa-flagged`, when per-type was still the default; `summarize.py` still reads that name. The render-only and per-type arms use copies of the plugin made at run time, so all four arms get exactly the same prompt. Each run starts in a fresh empty directory, so no arm can read the Rupa repo. `bench/summarize.py` prints one row per run and the medians per arm, with turns and cache reads first, then a line comparing each Rupa arm to the baseline.

First result (2026-10-15)

Three runs per arm, Claude Opus 5.5, task: "Write a 2-page PDF report on the AWS Well-Architected security pillar, saved as report.pdf." Every run produced a PDF. Medians:

Arm	Turns	Cache reads	Cost	vs baseline
`baseline`	9	299,095	$0.43	—
`rupa`	10	296,826	$0.40	turns +11%, cost −7%
`rupa-flagged`	13	438,591	$0.43	turns +44%, cost 0%
`rupa-verify` (per-type sheets)	15	523,338	$0.55	turns +67%, cost +27%

Rupa didn't save turns. The transcripts show why, and it isn't rendering:

- **Every Rupa run went over two pages on its first render** (three or four pages). The default contents list takes a page break, and Claude wrote a first draft longer than two pages. Baseline overshot too, but fixed it in one turn by editing its own CSS (`font-size`, margins). Rupa's form is fixed, so Claude cut content instead, probing each round with `pdftotext | wc -w`. One run took seven cut-and-render rounds.
- **The flagged arm threw its saving away.** On a clean PDF verify made no sheets, so Claude rendered the pages with `pdftoppm` and looked anyway.
- Baseline never detoured to find or install a PDF tool on this machine: Chrome and reportlab were already there. The estimate above assumed it would.

What changed in response: page budgets (above), so Rupa does the density fitting baseline did by hand and gives a word count when cutting is unavoidable; the skill sets `pages` when the user asks for a length, writes about 450 words per page, and doesn't look at pages when verify made no sheets; `sample: flagged` is now the default. A run of three per arm costs about $6 on Opus.

Second result (2026-10-16)

Same task, model and runs, after page budgets and flagged-only sampling. Every run produced a 2-page PDF. Medians:

Arm	Turns	Cache reads	Cost	vs baseline
`baseline`	10	351,496	$0.46	—
`rupa`	5	152,505	$0.27	turns −50%, cost −42%
`rupa-pertype`	9	247,766	$0.36	turns −10%, cost −22%
`rupa-verify` (as shipped)	6	153,765	$0.28	turns −40%, cost −39%

Every Rupa run set `pages: 2` and fitted on its first render, so none cut content or measured pages. The shipped arm is one render plus one `verify` that came back clean, with no sheets to open. Per-type sampling costs about three extra turns for sheets on a PDF that has nothing wrong with it, which confirms flagged as the default. The Rupa runs were also tighter: the slowest shipped run took 6 turns, against 20 before.

One caveat: the Rupa reports are shorter. Median text was about 850 words against about 1,180 for baseline, because baseline packs the page with smaller margins and type, and Rupa keeps the theme's margins. Some of the saving is less writing (median output 3,100 tokens against 7,150). For a fixed page count that's a fair trade, but it is a difference in the product as well as in cost.

Harder task (2026-10-16)

`TASK=guide`: a 3-page IAM Identity Center guide with a five-column table, a CLI example, a JSON policy of at least 25 lines and a numbered checklist. Three runs per arm, Opus 5.5. Every run produced a 3-page PDF with all four required parts. Medians:

Arm	Turns	Cache reads	Cost	vs baseline
`baseline`	20	786,720	$0.85	—
`rupa`	14	348,457	$0.47	turns −30%, cost −46%
`rupa-pertype`	17	312,611	$0.53	turns −15%, cost −38%
`rupa-verify` (as shipped)	7	249,016	$0.35	turns −65%, cost −59%

The gap widens with layout work. Baseline built the PDF with reportlab or Chrome, then went round after round adjusting page breaks and spacing and checking page images, writing 17,700 output tokens per run. The Rupa arms wrote less than half that.

The Rupa runs still spent turns cutting. The first draft was always over 3 pages at the tightest type, and the "cut about N words" estimate ran low: most runs took two to four cut rounds. Rupa counted the words on the overflow pages, but here the overflow was mostly the JSON policy, which has one or two words a line, so "cut 34 words" meant about fifteen lines of space. The estimate now measures the height used past the budget and converts it to prose words at tight type, plus 10%. On a guide-like sample it asked for 177 words, and one cut of that size fitted. The three-arm medians above predate this fix. The render-only arm happens to be slower than the shipped arm here (14 turns against 7); with three runs that's noise from how many cut rounds each run needed.

Rupa's guides again carry fewer words (about 750 against 1,300), for the same reason as before.

Harder task, with bundled fonts (2026-10-16)

Same guide task after the switch to bundled Inter and JetBrains Mono and the height-based estimate. Medians:

Arm	Turns	Cache reads	Cost	vs baseline
`baseline`	18	854,510	$0.94	—
`rupa`	7	204,375	$0.39	turns −61%, cost −58%
`rupa-pertype`	15	311,164	$0.53	turns −17%, cost −44%
`rupa-verify` (as shipped)	12	206,585	$0.42	turns −33%, cost −56%

Cost held, but cutting was still the weak spot. Every Rupa run went over 3 pages on its first render, and only 3 of the 6 render-only and shipped runs fitted after one cut. The runs that were told to cut a lot cut too much: they fitted at compact type with half a page spare, at 518 to 578 words. The cause was the unit. The hint was in words, but a code line or a table row frees several times the space of a prose word, so a run that cut code to meet a word count overshot. Claude also spent one turn per Edit.

What changed in response: the over-budget hint now asks for lines ("cut about 4 lines (a line of prose is about 18 words; a code line or table row is one line)"), and the skill plans about 30 lines per page before writing and makes all cuts in one turn.

Harder task, cuts in lines (2026-10-16)

Same task after the lines-based hint and the per-page line plan. Every run produced a 3-page PDF. Medians:

Arm	Turns	Cache reads	Cost	vs baseline
`baseline`	23	1,051,781	$1.19	—
`rupa`	5	107,903	$0.28	turns −78%, cost −76%
`rupa-pertype`	15	359,063	$0.49	turns −35%, cost −59%
`rupa-verify` (as shipped)	6	155,203	$0.31	turns −74%, cost −74%

The line plan mostly removed cutting. 7 of the 9 Rupa runs fitted on their first render, all three render-only runs took exactly 5 turns, and two of the three shipped runs were one render plus a clean verify. Baseline was costlier than in either earlier guide run ($1.19 against $0.85 and $0.94). Agent runs vary, so the percentages are wider than the true gap; the shipped arm's own cost ($0.31) is the steadier number.

Two runs still cut. One shipped run was asked to cut 4 lines, cut too little, was asked for 4 more and fitted: two rounds, with seven Edits in separate turns, so "all cuts in one turn" doesn't always hold. The skill now says how: rewrite the whole Markdown file with one Write, not a series of Edits, for budget cuts and verify fixes alike. The tight task below tests it. One per-type run fitted, then pushed itself back over the budget while fixing what it saw on the per-type sheets.

The plan errs short. Most runs fitted only once Rupa dropped to compact type, so they weren't over-cut, but the Rupa guides still carry about 740 words against baseline's 1,260. Baseline gets its extra words by shrinking margins and type: every baseline PDF fails `rupa verify`'s `overflow` and `text-color` checks.

One layout fault turned up in run 4's PDFs: a heading and a one-line lead-in ("Run from the delegated administrator account:") at the bottom of page 1, with the code block moved whole to page 2. Print CSS now keeps a paragraph with the code block or table that follows it and a heading's first list item with the second, and `heading-at-bottom` also flags a heading with one line directly under it at a page end.

Report task, current build (2026-10-16)

The 2-page report task again, with the bundled fonts, the line plan and the break rules above. Every run produced a 2-page PDF. Medians:

Arm	Turns	Cache reads	Cost	vs baseline
`baseline`	13	505,643	$0.52	—
`rupa`	5	152,505	$0.26	turns −62%, cost −50%
`rupa-pertype`	9	246,035	$0.34	turns −31%, cost −36%
`rupa-verify` (as shipped)	7	199,354	$0.29	turns −46%, cost −45%

All nine Rupa runs fitted on their first render, so none cut anything, and every Rupa PDF passes `rupa verify` while every baseline PDF fails `text-color`. The shipped arm cost about the same as in the second result ($0.29 against $0.28); two of its runs spent one extra turn listing the folder to confirm `report.pdf` was there. Rupa's reports came in at about 720 words against baseline's 1,180, down from about 850 in the second result: the line plan makes Claude write to the theme's density rather than past it.

Tight task: cuts in one Write (2026-10-16)

`TASK=tight`, the guide in two pages, so the run has to cut. Rupa arms only (`ARMS="rupa rupa-verify"`), three runs each, Opus 5.5, after the skill started telling Claude to cut by rewriting the whole file with one Write. Medians:

Arm	Turns	Cache reads	Cost
`rupa`	7	102,557	$0.33
`rupa-verify` (as shipped)	8	139,791	$0.36

Five of the six runs went over the budget on the first render, asking for cuts of 7 to 31 lines. All five cut with a single Write of the whole file and fitted on the next render: one cut round each, with no Edits in any run. The sixth fitted at first. Every PDF passes `rupa verify` and keeps all four required parts, with policies of about 29 to 35 lines. A run that has to cut costs two turns (a Write and a render) on top of a run that fits at once, which is all a cut should cost.

Wide task: shortening instead of splitting (2026-10-16)

`TASK=wide`, an EC2 sizing guide whose table must keep all eleven columns. One `rupa-verify` run, Opus 5.5: 16 turns, $0.70. The table overflowed; Claude shortened cell text for two fix rounds, still ran 13pt over, and reported a theme issue. It never split the table, though the skill listed that fix, and verify said only "text extends past the side margins". Now the `overflow` detail names what runs over and by how much (`p1 table by 13pt`), and the skill says to split an overflowing table into two that repeat the first column. One run: a failure mode, not a cost. After the change, a single run split the table on its first fix round and passed: 8 turns, $0.37.

Wide task, all arms (2026-10-16)

`TASK=wide`, three runs per arm, Opus 5.5, after the overflow and split changes. Medians:

Arm	Turns	Cost	PDFs passing verify
baseline	11	$0.54	0 of 3 (own margins and colors)
`rupa` (render only)	5	$0.31	0 of 3: table overflows
`rupa-pertype`	17	$0.64	2 of 3
`rupa-verify` (as shipped)	12	$0.47	3 of 3

Every Rupa first draft overflowed, by 31 to 85pt. Of the six runs that verified, two split the table on the first fix round and two shortened cells first and split on the second. Two only shortened: one fitted, and a per-type run stayed 25pt over and shipped with the failure reported. The runs that shortened first read "a table ... across eleven columns" as forbidding a split. Every fix round was a Python rewrite of the Markdown rather than the one Write the skill asks for, which cost nothing extra here.

This is the one task where baseline did well. It never clipped the table: two runs put it on a landscape page and one fitted it on portrait with half-inch margins, at 7.4pt and 6.6pt type (one header broke mid-word). Rupa can't change orientation or shrink one table, so it restructures instead and keeps 9.8pt type. That costs fix rounds: the shipped arm saved only 13% on cost and took one more turn. The render-only arm is cheapest but shipped every table overflowing, so on this task verify is what makes the output correct. A theme option for wide tables would cut the fix rounds; the next build adds one.

Wide tables in smaller type (2026-10-16)

`print.css` now sets a table of 9 or more columns in 8pt type, 11 or more in 7.5pt, with tighter cell padding; the old padding alone took about 160pt of an 11-column table's 497pt. The skill tells Claude

that about 11 short columns fit and to write two tables beyond that. Re-rendered with the new rule, all nine Rupa first drafts from the full wide run fit, where all nine had overflowed. One draft still fails verify on a separate problem: a 180-character `--query` in a code block, which Rupa wraps only at spaces so a command is never silently broken.

Measured with `TASK=wide ARMS="rupa rupa-verify"`, three runs each, Opus 5.5. Baseline is the full run above; it doesn't use Rupa, so the change can't affect it.

Arm	Turns	Cost	PDFs passing verify
baseline (full run above)	11	$0.54	0 of 3
`rupa` (render only)	5	$0.32	3 of 3
`rupa-verify` (as shipped)	6	$0.33	3 of 3

Every run fitted on its first render, with no fix rounds: one Write, one render, and for the shipped arm one clean verify. Each kept a single eleven-column table at 7.5pt, carried across a page break with its header repeated. Against baseline the shipped arm takes 45% fewer turns and costs 40% less, in line with the other tasks, where before the rule it saved 13%. Render-only now ships correct tables too, so on this task verify is a check that passes rather than the step that makes the output right.

How to read the results

- Compare medians, not single runs. Agent runs vary.
- Judge on turns and cost first. Output tokens are secondary.
- Count failed runs (no PDF) separately; a cheap run that produced nothing isn't a saving.
- Open one transcript per arm. Confirm the Rupa arms used Rupa and the render-only arm didn't verify. If an arm misbehaved, fix that before trusting any number.
- Run all arms on the same model on the same day, and pin the model with `MODEL=...`.

Well-Architected lens

The AWS Well-Architected pillars apply even to a local tool.

Pillar	How Rupa addresses it
Operational excellence	One status line and stable exit codes make it scriptable; CI runs tests and a smoke render on every push

Pillar	How Rupa addresses it
Security	Raw HTML escaped; PDF render limited to `file:` and `data:` URLs; Jinja autoescaping; no secrets anywhere
Reliability	Input validated before render; every failure maps to an exit code and one message; every PDF checked by `rupa verify`
Performance efficiency	No network, no browser process; this 30-plus-page document renders to PDF in a few seconds
Cost optimization and sustainability	Fewer turns and output tokens per document, measured rather than claimed; vision spent on sampled pages, not every page

Part 3: Tutorial

Before you start

You need:

- Python 3.11 or newer and uv.
- WeasyPrint's system libraries (Pango). On Ubuntu or Debian: `sudo apt install libpango-1.0-0 libpangoft2-1.0-0`. On macOS: `brew install pango`. On Windows, use WSL.
- Claude Code, for steps 7 to 9.

The finished project looks like this:

```
rupa/
├── pyproject.toml
├── README.md
├── src/rupa/
│   ├── __init__.py
│   ├── document.py
│   ├── render.py
│   ├── cli.py
│   ├── verify.py
│   ├── themes/
│   │   ├── default.css
│   │   └── print.css
│   └── references/
│       ├── sample.md     # source for every theme's reference pages
│       └── default/      # cover.png, prose.png, table.png, code.png
├── tests/
│   ├── test_document.py
│   ├── test_cli.py
│   └── test_verify.py
├── plugin/               # Claude Code plugin
├── wants.example.yaml
├── bench/
│   ├── bench.sh
│   ├── summarize.py
│   └── fake/claude     # stand-in for testing the plumbing
└── .github/workflows/ci.yml
```

Step 1: Scaffold the project

```
mkdir rupa && cd rupa
git init
mkdir -p src/rupa/themes tests plugin bench .github/workflows
```

Create `pyproject.toml`. The `[project.scripts]` entry is what makes `rupa` a command once installed. The `pythonpath` setting lets pytest import the package without installing it.

```
[project]
name = "rupa"
version = "0.1.0"
description = "Render Markdown + frontmatter from stdin into styled HTML or PDF."
readme = "README.md"
requires-python = "≥3.11"
dependencies = [
    "markdown-it-py≥3.0",
    "mdit-py-plugins≥0.4",
    "jinja2≥3.1",
    "pyyaml≥6.0",
    "pygments≥2.17",
    "weasyprint≥70",
    "pdfplumber≥0.11",
]

[project.scripts]
rupa = "rupa.cli:main"

[dependency-groups]
dev = ["pytest≥8"]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
packages = ["src/rupa"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]

[tool.ruff]
line-length = 88
```

Create `src/rupa/__init__.py`:

```
"""Rupa: a renderer layer. Content in on stdin, styled HTML or PDF out."""

__version__ = "0.1.0"
```

Then install everything into a local environment:

```
uv sync
```

Checkpoint: `uv run pytest` runs and reports that no tests were collected.

Step 2: The document model

This module turns text into a `Document`: metadata, body HTML and a list of headings. It knows nothing about styling. Note three choices that matter later:

- `html: False` escapes raw HTML in the input.
- Unknown frontmatter keys are an error, not silently ignored.
- Headings are collected from the token stream, which is what the table of contents uses.

`src/rupa/document.py`:

```
"""Parse Markdown + YAML frontmatter into a Document model.

The model is the contract between Claude and Rupa: Claude writes plain
Markdown, Rupa owns every styling decision downstream of this file.
"""

from __future__ import annotations

import re
from dataclasses import dataclass, field

import yaml
from markdown_it import MarkdownIt
from mdit_py_plugins.anchors import anchors_plugin
from mdit_py_plugins.footnote import footnote_plugin
from mdit_py_plugins.front_matter import front_matter_plugin
from mdit_py_plugins.tasklists import tasklists_plugin
from pygments import highlight
```

```
from pygments.formatters import HtmlFormatter
from pygments.lexers import TextLexer, get_lexer_by_name
from pygments.util import ClassNotFound

ALLOWED_META = {"title", "subtitle", "author", "date", "theme", "toc", "pages"}


class InputError(ValueError):
    """Raised when the input cannot be turned into a Document."""


def clip(value: object, limit: int = 80) -> str:
    """A value short enough to quote in a one-line error message."""
    text = " ".join(str(value).split())
    return text if len(text) <= limit else text[: limit - 3] + "..."


def yaml_problem(exc: Exception) -> str:
    # PyYAML's own message spans several lines with a source excerpt; keep
    # the problem and where it is.
    if isinstance(exc, yaml.MarkedYAMLError) and exc.problem:
        mark = exc.problem_mark
        where = f"line {mark.line + 1}, column {mark.column + 1}: " if mark else ""
        return where + exc.problem
    return clip(exc)


@dataclass
class Heading:
    level: int
    text: str
    anchor: str


@dataclass
class Document:
    meta: dict
    body_html: str
    headings: list[Heading] = field(default_factory=list)

    @property
    def title(self) -> str:
        if self.meta.get("title"):
            return str(self.meta["title"])
        return self.headings[0].text if self.headings else "Untitled"


def _highlight(code: str, lang: str, _attrs: str) -> str:
```

```
    try:
        lexer = get_lexer_by_name(lang) if lang else TextLexer()
    except ClassNotFound:
        lexer = TextLexer()
    return highlight(code, lexer, HtmlFormatter(nowrap=True))


def _build_parser() -> MarkdownIt:
    # html=False: raw HTML in the input is escaped, never passed through.
    md = MarkdownIt("commonmark", {"html": False, "highlight": _highlight}).enable(
        ["table", "strikethrough"]
    )
    md.use(front_matter_plugin)
    md.use(tasklists_plugin)
    md.use(footnote_plugin)
    md.use(anchors_plugin, min_level=1, max_level=3)
    return md


_SCHEME = re.compile(r"[a-zA-Z][a-zA-Z0-9+.-]*:")


def _check_image(src: str) -> None:
    """Images are relative paths or inline data, never fetched from elsewhere.

    A remote image in the HTML would be fetched by the reader's browser, which
    leaks that the page was opened and anything put in the URL.
    """
    if src.startswith("data:image/"):
        return
    if _SCHEME.match(src) or src.startswith(("/", "\\")):
        raise InputError(
            f"image '{clip(src)}' must be a relative path or a data:image URI"
        )


def _plain_text(inline) -> str:
    """Heading text without Markdown syntax: `**Big** `Plan`` → 'Big Plan'."""
    parts = []
    for child in inline.children or []:
        if child.type in ("text", "code_inline"):
            parts.append(child.content)
        elif child.type in ("softbreak", "hardbreak"):
            parts.append(" ")
    return "".join(parts).strip()


def parse(text: str) -> Document:
```

```
    if not text.strip():
        raise InputError("empty input: nothing to render")

    md = _build_parser()
    tokens = md.parse(text)

    meta: dict = {}
    headings: list[Heading] = []
    for i, tok in enumerate(tokens):
        if tok.type == "front_matter":
            try:
                loaded = yaml.safe_load(tok.content) or {}
            # An impossible date (2026-02-30) raises ValueError, and deeply
            # nested YAML RecursionError; both are bad input, not a crash.
            except (yaml.YAMLError, ValueError, OverflowError, RecursionError) as exc:
                raise InputError(f"bad frontmatter: {yaml_problem(exc)}") from exc
            if not isinstance(loaded, dict):
                raise InputError("frontmatter must be a YAML mapping")
            unknown = set(loaded) - ALLOWED_META
            if unknown:
                raise InputError(
                    f"unknown frontmatter keys: {clip(sorted(map(str, unknown)))}"
                )
            pages = loaded.get("pages")
            if pages is not None and (
                not isinstance(pages, int) or isinstance(pages, bool) or pages < 1
            ):
                raise InputError("frontmatter 'pages' must be a whole number ⩾ 1")
            meta = loaded
        elif tok.type == "heading_open":
            headings.append(
                Heading(
                    level=int(tok.tag[1]),
                    text=_plain_text(tokens[i + 1]),
                    anchor=tok.attrs.get("id", ""),
                )
            )

    for tok in tokens:
        for child in tok.children or []:
            if child.type == "image":
                _check_image(str(child.attrs.get("src", "")))

    doc = Document(meta=meta, body_html="", headings=headings)
    # The cover already shows the title, so a leading H1 that repeats it is
    # dropped from the body. Any other leading H1 is a real part and stays.
    first = next((i for i, t in enumerate(tokens) if t.type ≠ "front_matter"), None)
    if (
```

```
        first is not None
        and tokens[first].type == "heading_open"
        and tokens[first].tag == "h1"
        and _plain_text(tokens[first + 1]) == doc.title
    ):
        del tokens[first : first + 3]

    body = md.renderer.render(tokens, md.options, {})
    # Fenced code gets Pygments spans; mark the block for theme CSS.
    doc.body_html = re.sub(r"<pre><code", '<pre class="hl"><code', body)
    return doc
```

Write the tests before moving on. `tests/test_document.py`:

````
import pytest

from rupa.document import InputError, parse

SAMPLE = """---
title: Sample
author: Vajra Sarpa
---

## First section

Hello **world**.

```python
print("hi")
```
"""


def test_frontmatter_becomes_meta():
    doc = parse(SAMPLE)
    assert doc.meta == {"title": "Sample", "author": "Vajra Sarpa"}
    assert doc.title == "Sample"


def test_headings_are_collected_with_anchors():
    doc = parse(SAMPLE)
    assert [(h.level, h.text) for h in doc.headings] == [(2, "First section")]
    assert doc.headings[0].anchor == "first-section"


def test_code_is_highlighted():
````

```
    doc = parse(SAMPLE)
    assert '<pre class="hl">' in doc.body_html
    assert "<span" in doc.body_html


def test_title_falls_back_to_first_heading():
    assert parse("# From heading\n\ntext").title == "From heading"


def test_empty_input_is_rejected():
    with pytest.raises(InputError, match="empty"):
        parse("   \n")


def test_unknown_frontmatter_key_is_rejected():
    with pytest.raises(InputError, match="unknown frontmatter"):
        parse("---\ncolour: red\n---\n\ntext")


def test_raw_html_is_escaped():
    doc = parse("<script>alert(1)</script>\n")
    assert "<script>" not in doc.body_html
    assert "&lt;script&gt;" in doc.body_html


def test_non_mapping_frontmatter_is_rejected():
    with pytest.raises(InputError, match="mapping"):
        parse("---\n- a\n- b\n---\n\ntext")


def test_leading_h1_matching_the_title_is_dropped_from_the_body():
    doc = parse("---\ntitle: Report\n---\n\n# Report\n\ntext\n")
    assert "<h1" not in doc.body_html


def test_leading_h1_that_is_not_the_title_stays():
    doc = parse("---\ntitle: Report\n---\n\n# Background\n\ntext\n")
    assert '<h1 id="background">Background</h1>' in doc.body_html


def test_heading_text_has_no_markdown_syntax():
    doc = parse("# The **Big** `Plan`\n\ntext\n")
    assert doc.title == "The Big Plan"
    assert "<h1" not in doc.body_html  # it is the title, so the cover shows it


def test_unknown_keys_of_mixed_types_are_rejected_cleanly():
    with pytest.raises(InputError, match="unknown frontmatter"):
```

```
        parse("---\n1: x\nfoo: y\n---\n\ntext")


@pytest.mark.parametrize(
    "src", ["https://e.com/a.png", "//e.com/a.png", "/etc/a.png", "C:/a.png"]
)
def test_non_relative_images_are_rejected(src):
    with pytest.raises(InputError, match="relative path"):
        parse(f"![a]({src})\n")


def test_relative_and_data_images_are_allowed():
    doc = parse("![a](img/fig.png) ![b]()\n")
    assert 'src="img/fig.png"' in doc.body_html


@pytest.mark.parametrize("value", ["two", 0, -1, 1.5, True])
def test_pages_must_be_positive_int(value):
    with pytest.raises(InputError, match="pages"):
        parse(f"---\npages: {value}\n---\n\n# A\n")


def test_pages_is_allowed():
    assert parse("---\npages: 2\n---\n\n# A\n").meta["pages"] == 2


def test_strikethrough_task_lists_and_footnotes_render():
    doc = parse("- [x] done\n- [ ] todo\n\n~~old~~ new.[^1]\n\n[^1]: A <b>note</b>.\n")
    html = doc.body_html
    assert "<s>old</s>" in html
    assert html.count('type="checkbox"') == 2 and 'checked="checked"' in html
    assert 'class="footnotes"' in html and "[^1]" not in html
    assert "&lt;b&gt;note&lt;/b&gt;" in html  # raw HTML stays escaped in a footnote


@pytest.mark.parametrize(
    "front",
    ["date: 2026-02-30", "title: " + "[" * 3000],
    ids=["impossible-date", "deep-nesting"],
)
def test_frontmatter_that_crashes_yaml_is_bad_input(front):
    # ValueError (impossible date) and RecursionError (deep nesting) used to
    # escape as tracebacks with exit 1.
    with pytest.raises(InputError, match="bad frontmatter"):
        parse(f"---\n{front}\n---\n\nBody.\n")
```

Checkpoint: `uv run pytest tests/test_document.py -q` shows 8 passed.

Step 3: The renderer and themes

All styling decisions live in `render.py` and the CSS files. The HTML template is inline so the package has one less file to ship. The PDF function resolves relative images against the input file's folder and refuses to read files outside it.

One trap worth knowing: WeasyPrint's `FatalURLFetchingError` inherits from `BaseException`, not `Exception`, so a plain `except Exception` does not catch it. That is why the code names it explicitly. This was found by a failing test during the build.

`src/rupa/render.py` :

```
"""Turn a Document into HTML or PDF bytes. All styling lives here and in themes/."""

from __future__ import annotations

import base64
import html as html_lib
import re
from dataclasses import dataclass
from importlib import resources
from pathlib import Path
from urllib.parse import unquote, unquote_to_bytes, urlparse

from jinja2 import Environment, select_autoescape
from pygments.formatters import HtmlFormatter

from .document import Document, clip

TEMPLATE = """<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta name="keywords" content="rupa-theme:{{ theme }}">
<title>{{ title }}</title>
<style>{{ css | safe }}</style>
</head>
<body class="{{ target }}">
<header class="cover">
  <h1 class="doc-title">{{ title }}</h1>
  {% if meta.subtitle %}<p class="subtitle">{{ meta.subtitle }}</p>{% endif %}
  {% if meta.author or meta.date %}
  <p class="byline">{{ meta.author or "" }}{% if meta.author and meta.date %} &middot; {%
endif %}{{ meta.date or "" }}</p>
```

```
  {% endif %}
</header>
{% if toc %}
<nav class="toc"><h2>Contents</h2><ol>
{% for h in toc %}<li class="l{{ h.level }}"><a href="#{{ h.anchor }}">{{ h.text }}</a></li>
{% endfor %}</ol></nav>
{% endif %}
<main>
{{ body | safe }}
</main>
</body>
</html>
"""

THEMES = ("default", "cyan", "cyan-dark")
# A theme can share another's screen CSS and differ only in its print layer.
THEME_BASE = {"cyan-dark": "cyan"}
# The text colours verify accepts in a theme's PDFs. Light themes print solid
# black; a dark theme prints its own ink, accent and secondary greys.
THEME_TEXT_COLORS = {
    "default": ("#000000",),
    "cyan": ("#000000",),
    "cyan-dark": ("#e6edf6", "#38bdf8", "#9fb0c8", "#6b7d98"),
}

# Fonts ship with Rupa so a PDF looks the same on every machine, whatever is
# installed. Only the PDF target uses them; HTML keeps the CSS font stack.
FONT_DIR = Path(str(resources.files("rupa") / "themes" / "fonts")).resolve()
FONT_FACES = (  # (family, file stem, weight, style)
    ("Inter", "Inter-Regular", 400, "normal"),
    ("Inter", "Inter-Italic", 400, "italic"),
    ("Inter", "Inter-Medium", 500, "normal"),
    ("Inter", "Inter-SemiBold", 600, "normal"),
    ("Inter", "Inter-Bold", 700, "normal"),
    ("Inter", "Inter-BoldItalic", 700, "italic"),
    ("Hanken Grotesk", "HankenGrotesk-Regular", 400, "normal"),
    ("Hanken Grotesk", "HankenGrotesk-Italic", 400, "italic"),
    ("Hanken Grotesk", "HankenGrotesk-Medium", 500, "normal"),
    ("Hanken Grotesk", "HankenGrotesk-SemiBold", 600, "normal"),
    ("Hanken Grotesk", "HankenGrotesk-Bold", 700, "normal"),
    ("Hanken Grotesk", "HankenGrotesk-BoldItalic", 700, "italic"),
    ("JetBrains Mono", "JetBrainsMono-Regular", 400, "normal"),
    ("JetBrains Mono", "JetBrainsMono-Bold", 700, "normal"),
)


def _font_faces() -> str:
    return "\n".join(
```

```
        f'@font-face {{ font-family: "{family}"; font-weight: {weight}; '
        f'font-style: {style}; src: url("{(FONT_DIR / f"{stem}.ttf").as_uri()}"); }}'
        for family, stem, weight, style in FONT_FACES
    )


class RenderError(RuntimeError):
    """Raised when a target cannot be produced."""


def _theme_css(theme: str, target: str) -> str:
    if theme not in THEMES:
        raise RenderError(f"unknown theme '{clip(theme)}' (have: {', '.join(THEMES)})")
    pkg = resources.files("rupa") / "themes"
    css = (pkg / f"{THEME_BASE.get(theme, theme)}.css").read_text(encoding="utf-8")
    css += "\n" + HtmlFormatter(style="default").get_style_defs(".hl")
    if target == "pdf":
        # print.css goes last so it overrides Pygments' unscoped `pre` rule too,
        # then the theme's own print layer, if it has one.
        css = _font_faces() + "\n" + css
        css += "\n" + (pkg / "print.css").read_text(encoding="utf-8")
        layer = pkg / f"{theme}.print.css"
        if layer.is_file():
            css += "\n" + layer.read_text(encoding="utf-8")
    return css


# Steps for fitting a page budget, tried in order after the theme as written.
# Each changes only type size and spacing: page margins stay put (verify
# measures the body box from print.css) and headings stay at or above 11.5pt
# (verify detects headings by size).
FIT_STEPS = (
    ("no contents", None),
    (
        "compact type",
        """
body { font-size: 9.75pt; line-height: 1.42; }
.cover { margin-bottom: 1.1rem; padding-bottom: .8rem; }
h1 { margin: 1.8rem 0 .7rem; }
h2 { margin: 1.5rem 0 .55rem; }
h3 { margin: 1.1rem 0 .4rem; }
p, ul, ol { margin: 0 0 .65rem; }
li { margin: .1rem 0; }
table { margin: 0 0 .8rem; font-size: 8.75pt; }
th, td { padding: .3rem .5rem; }
pre { padding: .6rem .8rem; line-height: 13.12pt; }
""",
    ),
```

```
    (
        "tight type",
        """
body { font-size: 9pt; line-height: 1.32; }
.cover { margin-bottom: .8rem; padding-bottom: .6rem; }
h1 { margin: 1.2rem 0 .5rem; }
h2 { margin: 1rem 0 .4rem; padding-bottom: .2rem; }
h3 { margin: .8rem 0 .3rem; }
p, ul, ol { margin: 0 0 .45rem; }
li { margin: 0; }
table { margin: 0 0 .6rem; font-size: 8.25pt; }
th, td { padding: .2rem .4rem; }
pre { padding: .45rem .6rem; font-size: 7.8pt; line-height: 12.15pt; }
""",
    ),
)


_IMG_SRC = re.compile(r'(<img src=")([^"]+)(")')
# Raster images by the format Pillow detects from the bytes, not the name.
RASTER_MIME = {
    "PNG": "image/png",
    "JPEG": "image/jpeg",
    "GIF": "image/gif",
    "WEBP": "image/webp",
}
IMAGE_SUFFIXES = (".png", ".jpg", ".jpeg", ".gif", ".webp", ".svg")


def _decoded_mime(data: bytes, name: str) -> str:
    """The image's MIME type, once its bytes are known to decode as an image.

    A file that only claims to be an image (a renamed .env, a truncated PNG)
    fails here, so it is neither embedded in HTML nor dropped from a PDF.
    """
    if name.lower().endswith(".svg"):
        import xml.etree.ElementTree as ET

        try:
            root = ET.fromstring(data)
        except ET.ParseError as exc:
            raise RenderError(f"image {name} is not valid SVG: {exc}") from exc
        if root.tag not in ("svg", "{http://www.w3.org/2000/svg}svg"):
            raise RenderError(f"image {name} is not an SVG image")
        return "image/svg+xml"

    import io
    import warnings
```

```
    from PIL import Image, ImageFile

    # Importing WeasyPrint turns this on for the whole process, which would
    # let half a file through as an image.
    truncated_ok, ImageFile.LOAD_TRUNCATED_IMAGES = (
        ImageFile.LOAD_TRUNCATED_IMAGES,
        False,
    )
    try:
        with warnings.catch_warnings():
            warnings.simplefilter("error", Image.DecompressionBombWarning)
            with Image.open(io.BytesIO(data)) as img:
                fmt = img.format
                img.load()
    except Exception as exc:  # Pillow raises many types on bad data
        raise RenderError(f"image {name} cannot be decoded: {exc}") from exc
    finally:
        ImageFile.LOAD_TRUNCATED_IMAGES = truncated_ok
    if fmt not in RASTER_MIME:
        raise RenderError(f"image {name} is {fmt}; use PNG, JPEG, GIF, WebP or SVG")
    return RASTER_MIME[fmt]


def load_image(src: str, base_dir: str | Path) -> tuple[bytes, str]:
    """Read one image the document names: (bytes, MIME type).

    A data: URI is decoded; a relative path must be an image file inside
    base_dir, not hidden (no part of it starts with '.'). Anything missing,
    refused or undecodable raises RenderError, so the render fails instead
    of leaving a broken image or embedding a file that isn't one.
    """
    if src.startswith("data:"):
        header, _, payload = src.partition(",")
        try:
            data = (
                base64.b64decode(payload, validate=True)
                if header.endswith(";base64")
                else unquote_to_bytes(payload)
            )
        except ValueError as exc:
            raise RenderError(f"image data URI cannot be decoded: {exc}") from exc
        return data, _decoded_mime(data, "data URI")

    base = Path(base_dir).resolve()
    path = (base / unquote(src)).resolve()
    if not path.is_relative_to(base):
        raise RenderError(f"blocked or missing resource: {path} is outside {base}")
```

```
    if any(part.startswith(".") for part in path.relative_to(base).parts):
        raise RenderError(f"blocked resource: {path} is a hidden file")
    if path.suffix.lower() not in IMAGE_SUFFIXES:
        raise RenderError(
            f"blocked resource: {path.name} is not an image"
            f" ({', '.join(IMAGE_SUFFIXES)})"
        )
    try:
        data = path.read_bytes()
    except OSError as exc:
        raise RenderError(f"blocked or missing resource: {path}") from exc
    return data, _decoded_mime(data, path.name)


def embed_images(body: str, base_dir: str | Path) -> str:
    """Inline every image as a data: URI, so the page needs no other file.

    Each one goes through load_image first: only real images, only from
    inside base_dir.
    """

    def inline(m: re.Match) -> str:
        data, mime = load_image(html_lib.unescape(m.group(2)), base_dir)
        uri = f"data:{mime}base64,{base64.b64encode(data).decode('ascii')}"
        return m.group(1) + uri + m.group(3)

    return _IMG_SRC.sub(inline, body)


def to_html(
    doc: Document,
    theme: str = "default",
    target: str = "html",
    extra_css: str = "",
    toc_on: bool | None = None,
    base_dir: str | Path | None = None,
) -> str:
    """Return the page as HTML. With base_dir, images are embedded (see embed_images)."""
    env = Environment(autoescape=select_autoescape(default=True))
    if toc_on is None:
        toc_on = doc.meta.get("toc", True)
    # H1s act as parts, H2s as sections. An H1 that repeats the title is skipped.
    toc = (
        [h for h in doc.headings if h.level <= 2 and h.text != doc.title]
        if toc_on
        else []
    )
    return env.from_string(TEMPLATE).render(
```

```
        title=doc.title,
        meta=doc.meta,
        toc=toc,
        body=embed_images(doc.body_html, base_dir) if base_dir else doc.body_html,
        css=_theme_css(theme, target) + extra_css,
        target=target,
        theme=theme,
    )


def to_pdf(
    doc: Document,
    theme: str = "default",
    base_dir: str | Path | None = None,
    extra_css: str = "",
    toc_on: bool | None = None,
) -> tuple[bytes, int]:
    """Return (pdf_bytes, page_count).

    Relative image paths resolve against base_dir (default: the current
    directory), and only files inside it can be read.
    """
    try:
        from weasyprint import HTML, URLFetcher
        from weasyprint.urls import FatalURLFetchingError
    except ImportError as exc:  # pragma: no cover - depends on system libs
        raise RenderError("PDF needs WeasyPrint and its system libraries") from exc

    base = Path(base_dir or ".").resolve()

    class ContainedFetcher(URLFetcher):
        def fetch(self, url, headers=None):
            if url.startswith("file:"):
                path = Path(unquote(urlparse(url).path)).resolve()
                if not (path.is_relative_to(base) or path.parent == FONT_DIR):
                    raise ValueError(f"{path} is outside {base}")
            return super().fetch(url, headers)

    # Files under base, Rupa's own fonts, and data: URIs only. Anything else,
    # or a missing file, fails the render instead of fetching from the
    # network or vanishing. Images arrive already checked and inlined, since
    # WeasyPrint swaps an image it can't decode for its alt text.
    fetcher = ContainedFetcher(allowed_protocols={"file", "data"}, fail_on_errors=True)
    html = to_html(
        doc, theme, target="pdf", extra_css=extra_css, toc_on=toc_on, base_dir=base
    )
    try:
        rendered = HTML(
```

```
            string=html, base_url=f"{base.as_uri()}/", url_fetcher=fetcher
        ).render()
        return rendered.write_pdf(), len(rendered.pages)
    except FatalURLFetchingError as exc:  # subclasses BaseException, not Exception
        raise RenderError(f"blocked or missing resource: {exc}") from exc
    except Exception as exc:  # keep the exit-4 contract for any WeasyPrint failure
        raise RenderError(f"PDF render failed: {exc}") from exc


@dataclass
class Fit:
    data: bytes
    pages: int
    steps: list[str]  # fit steps applied, in order; empty if none were needed
    cut_lines: int  # lines to cut to fit, at the last step; 0 when it fits


# At the tightest fit step a line takes about 12.4pt of page height, paragraph
# gaps included, and a line of prose holds about 18 words (measured on full
# pages of body text and of code).
TIGHT_LINE_PT = 12.4
PROSE_WORDS_PER_LINE = 18


def _lines_after(data: bytes, budget: int) -> int:
    """Lines of content to cut so the space used past the budget fits.

    Counted in lines, not words: overflow is often code or a table, where a
    line holds one or two words, so a word count asks for far too little
    there and far too much when prose is cut instead.
    """
    import io
    import math

    import pdfplumber

    from .verify import FOOTER_CUTOFF, MARGIN_TOP, PT_PER_IN

    used = 0.0
    with pdfplumber.open(io.BytesIO(data)) as pdf:
        for page in pdf.pages[budget:]:
            footer = page.height - FOOTER_CUTOFF * PT_PER_IN
            body = [c["bottom"] for c in page.chars if c["bottom"] < footer]
            if body:
                used += max(body) - MARGIN_TOP * PT_PER_IN
    # 10% over, since blocks that can't split (rows, short code) break late.
    return math.ceil(used / TIGHT_LINE_PT * 1.1)
```

```
def fit_pdf(
    doc: Document,
    theme: str = "default",
    base_dir: str | Path | None = None,
    budget: int | None = None,
) → Fit:
    """Render a PDF, tightening it step by step until it fits `budget` pages.

    Content is never cut. If the tightest step is still over, the result
    says roughly how many lines to cut to free the space used past the
    budget.
    """
    data, pages = to_pdf(doc, theme, base_dir)
    if not budget or pages ⩽ budget:
        return Fit(data, pages, [], 0)
    applied: list[str] = []
    css = ""
    toc_on = doc.meta.get("toc", True)
    for name, step_css in FIT_STEPS:
        if step_css is None:
            if not toc_on:
                continue
            toc_on = False
        else:
            css = step_css
        applied.append(name)
        data, pages = to_pdf(doc, theme, base_dir, extra_css=css, toc_on=toc_on)
        if pages ⩽ budget:
            return Fit(data, pages, applied, 0)
    return Fit(data, pages, applied, _lines_after(data, budget))
```

The screen theme, `src/rupa/themes/default.css`:

```
/* Rupa default theme: slate ink, one cyan accent. Screen first. */
:root {
  --ink: #0f172a;
  --muted: #334155;
  --rule: #cbd5e1;
  --accent: #0e7490;
  --code-bg: #f1f5f9;
  --bg: #ffffff;
  --measure: 46rem;
}
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
```

```
    --ink: #e2e8f0; --muted: #94a3b8; --rule: #334155;
    --accent: #22d3ee; --code-bg: #1e293b; --bg: #0f172a;
  }
}
* { box-sizing: border-box; }
html { background: var(--bg); }
body {
  margin: 0 auto;
  padding: 3rem 1rem 5rem;
  max-width: var(--measure);
  font: 400 16px/1.65 Inter, "Helvetica Neue", Arial, sans-serif;
  color: var(--ink);
  background: var(--bg);
}

/* Cover and table of contents */
.cover {
  border-bottom: 3px solid var(--accent);
  padding-bottom: 1.25rem;
  margin-bottom: 2rem;
}
.doc-title {
  font-size: 2.3rem;
  line-height: 1.15;
  margin: 0 0 .4rem;
  letter-spacing: -0.02em;
}
.subtitle { font-size: 1.15rem; color: var(--muted); margin: 0 0 .6rem; }
.byline { font-size: .9rem; color: var(--muted); margin: 0; }
.toc {
  margin: 0 0 2.5rem;
  padding: 1rem 1.25rem;
  border: 1px solid var(--rule);
  border-radius: 8px;
}
.toc h2 {
  font-size: .8rem;
  text-transform: uppercase;
  letter-spacing: .08em;
  margin: 0 0 .5rem;
  border: 0;
  padding: 0;
}
.toc ol { margin: 0; padding-left: 1.25rem; }
.toc a { color: var(--ink); text-decoration: none; }
.toc li.l1 {
  font-weight: 600;
  margin-top: .5rem;
```

```
  list-style: none;
  margin-left: -1.25rem;
}
.toc li.l2 { list-style: none; }
.toc a:hover { color: var(--accent); }

/* Headings and body */
h1, h2, h3 { line-height: 1.25; letter-spacing: -0.01em; }
h1 { font-size: 1.9rem; margin: 3rem 0 1rem; }
h2 {
  font-size: 1.45rem;
  margin: 2.75rem 0 .9rem;
  padding-bottom: .35rem;
  border-bottom: 1px solid var(--rule);
}
h3 { font-size: 1.12rem; margin: 2rem 0 .6rem; }
p, ul, ol { margin: 0 0 1rem; }
li { margin: .2rem 0; }
a { color: var(--accent); }
strong { font-weight: 600; }
blockquote {
  margin: 1.25rem 0;
  padding: .6rem 1rem;
  border-left: 4px solid var(--accent);
  background: var(--code-bg);
}
blockquote p:last-child { margin-bottom: 0; }

/* Code */
code {
  font: 0.88em/1.5 "JetBrains Mono", "DejaVu Sans Mono", Menlo, monospace;
  background: var(--code-bg);
  padding: .1em .35em;
  border-radius: 4px;
}
pre {
  background: var(--code-bg);
  padding: .9rem 1rem;
  border-radius: 8px;
  overflow-x: auto;
  border: 1px solid var(--rule);
}
pre code { background: none; padding: 0; font-size: .82rem; }

/* Tables */
table {
  border-collapse: collapse;
  width: 100%;
```

```
  margin: 0 0 1.25rem;
  font-size: .92rem;
  display: block;
  overflow-x: auto;
}
th, td {
  border: 1px solid var(--rule);
  padding: .45rem .6rem;
  text-align: left;
  vertical-align: top;
}
th { background: var(--code-bg); font-weight: 600; }
hr { border: 0; border-top: 1px solid var(--rule); margin: 2.5rem 0; }

/* Task lists and footnotes */
.contains-task-list { list-style: none; padding-left: 0; }
.task-list-item-checkbox { display: inline-block; margin: 0 .5em 0 0; vertical-align: -.1em; }
hr.footnotes-sep { margin: 2rem 0 1rem; }
.footnotes { font-size: .88em; }
.footnote-ref a { text-decoration: none; }
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) .hl * { color: inherit !important; }
}
```

The PDF overrides, `src/rupa/themes/print.css`. This is where text is forced to #000000:

```
/* PDF overrides. Text is forced solid black (#000000, full opacity). */
@page {
  size: Letter;
  margin: 0.85in 0.8in 0.9in;
  @bottom-right {
    content: counter(page) " / " counter(pages);
    font: 500 8.5pt Inter, sans-serif;
    color: #000000;
  }
  @bottom-left {
    content: string(doctitle);
    font: 500 8.5pt Inter, sans-serif;
    color: #000000;
  }
}
@page :first { @bottom-left { content: none; } }
:root {
  --ink: #000000; --muted: #000000; --accent: #0e7490;
  --rule: #94a3b8; --code-bg: #f1f5f9; --bg: #ffffff;
}
```

```
html, body { background: #ffffff; }
body {
  max-width: none;
  padding: 0;
  font-size: 10.5pt;
  line-height: 1.55;
  color: #000000;
  font-weight: 400;
}
body, p, li, td, th, h1, h2, h3, h4, blockquote, code, pre,
.subtitle, .byline, .toc a, a {
  color: #000000 !important;
  opacity: 1 !important;
}
.hl * { color: #000000 !important; font-style: normal !important; }

.doc-title { string-set: doctitle content(); font-weight: 700; font-size: 26pt; }
h1 { font-weight: 700; font-size: 20pt; }
h2 { font-weight: 700; font-size: 15pt; break-after: avoid; }
h3 { font-weight: 600; font-size: 12pt; break-after: avoid; }
strong, th { font-weight: 700; }
.subtitle, .byline { font-weight: 500; }
a { text-decoration: none; }

.toc { break-after: page; font-size: 9pt; line-height: 1.35; padding: .6rem 1rem; }
.toc li { font-weight: 500; margin: .05rem 0; }
.toc li.l1 { margin-top: .35rem; }
main h1 { break-before: page; margin-top: 0; }

blockquote, tr { break-inside: avoid; }
thead { display: table-header-group; }
pre { break-inside: auto; white-space: pre-wrap; orphans: 3; widows: 3; } /* wrap at spaces,
never mid-token; no 1-2 line fragments across a page break */
/* pre carries the code font and a fixed line height, so each line box is
   exactly that tall; with Inter as the block's strut, the height came from
   mixing two fonts' metrics and differed between Pango builds. */
pre { font: 8.3pt/13.875pt "JetBrains Mono", "DejaVu Sans Mono", monospace; }
pre code { font-size: 1em; line-height: inherit; }
p:not(:has(> img)):is(:has(+ pre), :has(+ table)) { break-after: avoid; } /* a lead-in ("Run
this:") moves with its block, and so does the heading above it */
p:has(> img) { break-before: avoid; } /* and with its image; WeasyPrint has no p:has(+ p >
img) */
:is(h2, h3) + :is(ul, ol) > li:first-child { break-after: avoid; } /* a heading never ends a
page with one list item */
table { display: table; font-size: 9pt; }
.footnote-backref { display: none; } /* a link back up the page does nothing on paper */
/* Wide tables get smaller type and tighter cells, so up to about 11 columns
   fit the text width instead of running into the margin. Specific enough to
```

```
   beat the fit steps' table and cell rules. */
table:has(th:nth-child(9)) { font-size: 8pt; }
table:has(th:nth-child(11)) { font-size: 7.5pt; }
table:has(th:nth-child(9)) :is(th, td) { padding: .25rem .3rem; }
```

Checkpoint: in `uv run python`, run `from rupa.document import parse; from rupa.render import to_pdf; data, n = to_pdf(parse("## Hi")); print(n)`. It prints a page count.

Step 4: The CLI contract

The CLI is where the contract from Part 2 becomes code. Notice that the success path prints exactly one line, and that every failure path has its own exit code.

This listing already includes the `verify` and `refs` subcommands from Step 8. They import `verify.py` only when called, so the CLI works before that file exists.

`src/rupa/cli.py`:

```
"""rupa CLI.

  rupa [INPUT] -o OUT            render Markdown to HTML or PDF
  rupa verify PDF [--wants F]    check a rendered PDF, write review sheets
  rupa refs [--theme T]          rebuild a theme's reference pages

A Markdown file named verify or refs is read as the subcommand; write
./verify to render it.

Contract:
  stdin  → Markdown with optional YAML frontmatter
  file   → the rendered output (format from -o extension or --to)
  stdout → ONE status line (or one JSON object with --json). Never the document.
  exit   → 0 ok, 2 usage, 3 bad input, 4 render failure,
           5 verify checks failed, or a PDF still over its page budget

Every run that exits non-zero or crashes is appended to a JSONL log,
~/.rupa/log.jsonl by default. RUPA_LOG=<path> moves it; RUPA_LOG=off stops it.
"""

from __future__ import annotations

import argparse
import json
import os
import sys
```

```
import tempfile
import traceback
from datetime import UTC, datetime
from importlib import resources
from pathlib import Path

from . import __version__
from .document import InputError, parse
from .render import (
    PROSE_WORDS_PER_LINE,
    THEMES,
    RenderError,
    fit_pdf,
    to_html,
    to_pdf,
)

EXIT_OK, EXIT_USAGE, EXIT_INPUT, EXIT_RENDER, EXIT_CHECKS = 0, 2, 3, 4, 5
FORMATS = ("html", "pdf")


def _render_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(
        prog="rupa",
        description="Render Markdown to HTML or PDF.",
        epilog="To render a file named verify or refs, write ./verify or ./refs.",
    )
    p.add_argument(
        "input", nargs="?", default="-", help="Markdown file, or - for stdin (default)"
    )
    p.add_argument("-o", "--output", required=True, help="output path, e.g. out.pdf")
    p.add_argument(
        "--to", choices=FORMATS, help="force format instead of using the extension"
    )
    p.add_argument(
        "--theme", default=None, help="theme name (default: frontmatter or 'default')"
    )
    p.add_argument(
        "--pages",
        type=int,
        help="PDF page budget; tighten type to fit (default: frontmatter 'pages')",
    )
    p.add_argument(
        "--force",
        action="store_true",
        help="with --to, replace an existing output that isn't .html or .pdf",
    )
    p.add_argument("--json", action="store_true", help="print a JSON status object")
```

```
    p.add_argument("--version", action="version", version=f"rupa {__version__}")
    return p


def _err(args, msg: str) -> None:
    msg = " ".join(msg.split())  # the contract is one line on stderr
    if getattr(args, "json", False):
        print(json.dumps({"ok": False, "error": msg}))
    else:
        print(f"rupa: error: {msg}", file=sys.stderr)


def render_main(argv: list[str]) -> int:
    args = _render_parser().parse_args(argv)
    out = Path(args.output)
    fmt = args.to or out.suffix.lstrip(".").lower()
    if fmt not in FORMATS:
        _err(args, f"cannot infer format from '{out.name}'; use --to")
        return EXIT_USAGE
    # --to lets -o name any file, so only an .html or .pdf is replaced unasked.
    if out.exists() and out.suffix.lower() not in (".html", ".pdf") and not args.force:
        _err(args, f"{out} exists and isn't .html or .pdf; use --force to replace it")
        return EXIT_USAGE

    try:
        # Bytes, decoded here: without a UTF-8 locale, text-mode stdin would
        # let bad bytes through as surrogates that fail later in the render.
        raw = (
            sys.stdin.buffer.read()
            if args.input == "-"
            else Path(args.input).read_bytes()
        )
        text = raw.decode("utf-8")
        doc = parse(text)
    except (InputError, OSError, UnicodeDecodeError) as exc:
        _err(args, str(exc))
        return EXIT_INPUT

    if args.pages is not None and args.pages < 1:
        _err(args, "--pages must be 1 or more")
        return EXIT_USAGE
    theme = args.theme or doc.meta.get("theme", "default")
    budget = args.pages or doc.meta.get("pages")
    try:
        pages, fit = None, None
        base = Path(args.input).parent if args.input != "-" else Path.cwd()
        if fmt == "html":
            data = to_html(doc, theme, base_dir=base).encode("utf-8")
```

```
        else:
            fit = fit_pdf(doc, theme, base_dir=base, budget=budget)
            data, pages = fit.data, fit.pages
        out.parent.mkdir(parents=True, exist_ok=True)
        out.write_bytes(data)
    except (RenderError, OSError) as exc:
        _err(args, str(exc))
        return EXIT_RENDER

    over = bool(fit and fit.cut_lines)
    if args.json:
        status = {
            "ok": not over,
            "path": str(out),
            "format": fmt,
            "pages": pages,
            "bytes": len(data),
        }
        if fit and budget:
            status.update(budget=budget, fit=fit.steps, cut_lines=fit.cut_lines)
        print(json.dumps(status))
    else:
        extra = f", {pages} page{'s' if pages != 1 else ''}" if pages else ""
        if fit and fit.steps and not over:
            extra += f", fit to {budget} with {' + '.join(fit.steps)}"
        line = f"rupa: wrote {out} ({fmt}{extra}, {len(data):,} bytes)"
        if over:
            line += (
                f"; over the {budget}-page budget at the tightest type:"
                f" cut about {fit.cut_lines} lines (a line of prose is about"
                f" {PROSE_WORDS_PER_LINE} words; a code line or table row is"
                " one line) and render again"
            )
        print(line)
    return EXIT_CHECKS if over else EXIT_OK


def verify_main(argv: list[str]) -> int:
    from .verify import (
        EXIT_CHECKS_FAILED,
        SAMPLE_MODES,
        PdfError,
        WantsError,
        load_wants,
        verify,
    )

    p = argparse.ArgumentParser(prog="rupa verify", description="Check a rendered PDF.")
```

```
    p.add_argument("pdf")
    p.add_argument("--wants", help="wants.yaml (default: built-in rules)")
    p.add_argument(
        "--theme",
        choices=THEMES,
        help="theme whose reference pages to compare against",
    )
    p.add_argument(
        "--out", help="directory for sheets and verify.json (default: <pdf>.verify/)"
    )
    p.add_argument(
        "--sample",
        choices=SAMPLE_MODES,
        help="which pages get review sheets (overrides the wants file)",
    )
    p.add_argument("--json", action="store_true")
    args = p.parse_args(argv)
    pdf = Path(args.pdf)
    if not pdf.is_file():
        _err(args, f"no such file: {pdf}")
        return EXIT_INPUT
    try:
        wants = load_wants(args.wants)
    except WantsError as exc:
        _err(args, str(exc))
        return EXIT_INPUT
    if args.sample:
        wants["vision"]["sample"] = args.sample
    out = Path(args.out) if args.out else pdf.with_suffix(".verify")
    try:
        report = verify(pdf, wants, out, theme=args.theme)
    except PdfError as exc:
        _err(args, str(exc))
        return EXIT_INPUT
    if args.json:
        print(Path(report.manifest).read_text())
    else:
        print(report.summary())
    return EXIT_OK if report.ok else EXIT_CHECKS_FAILED


def refs_main(argv: list[str]) -> int:
    from .verify import build_references

    p = argparse.ArgumentParser(
        prog="rupa refs", description="Rebuild a theme's reference pages."
    )
    p.add_argument("--theme", default="default")
```

```
    p.add_argument(
        "--out", help="output directory (default: the package's references/<theme>/)"
    )
    args = p.parse_args(argv)
    sample = (resources.files("rupa") / "references" / "sample.md").read_text(
        encoding="utf-8"
    )
    try:
        data, _ = to_pdf(parse(sample), args.theme)
    except RenderError as exc:
        _err(args, str(exc))
        return EXIT_RENDER
    out = (
        Path(args.out)
        if args.out
        else Path(str(resources.files("rupa") / "references" / args.theme))
    )
    with tempfile.TemporaryDirectory() as tmp:
        pdf = Path(tmp) / "sample.pdf"
        pdf.write_bytes(data)
        try:
            picked = build_references(pdf, out)
        except ValueError as exc:
            _err(args, str(exc))
            return EXIT_RENDER
    kinds = ", ".join(f"{k}=p{n}" for k, n in sorted(picked.items()))
    print(f"rupa: wrote references for '{args.theme}' to {out} ({kinds})")
    return EXIT_OK


class _Tee:
    """Pass writes through to a stream and keep a copy for the log."""

    def __init__(self, stream):
        self.stream, self.text = stream, []

    def write(self, s):
        self.text.append(s)
        return self.stream.write(s)

    def __getattr__(self, name):
        return getattr(self.stream, name)


def _log_path() -> Path | None:
    value = os.environ.get("RUPA_LOG", "")
    if value.lower() in ("off", "0", "false", "no"):
        return None
```

```
    return Path(value) if value else Path.home() / ".rupa" / "log.jsonl"


def _log(argv, code, out: _Tee, err: _Tee, crash: str | None = None) -> None:
    path = _log_path()
    if path is None:
        return
    entry = {
        "time": datetime.now(UTC).isoformat(timespec="seconds"),
        "version": __version__,
        "cwd": str(Path.cwd()),
        "argv": argv,
        "exit": code,
        "stdout": "".join(out.text).strip(),
        "stderr": "".join(err.text).strip(),
    }
    if crash:
        entry["traceback"] = crash
    try:
        path.parent.mkdir(parents=True, exist_ok=True)
        with path.open("a", encoding="utf-8") as f:
            f.write(json.dumps(entry) + "\n")
    except OSError:
        pass  # logging must never change what the CLI does


def _dispatch(argv: list[str]) -> int:
    if argv and argv[0] == "verify":
        return verify_main(argv[1:])
    if argv and argv[0] == "refs":
        return refs_main(argv[1:])
    return render_main(argv)


def main(argv: list[str] | None = None) -> int:
    argv = list(sys.argv[1:] if argv is None else argv)
    out, err = _Tee(sys.stdout), _Tee(sys.stderr)
    sys.stdout, sys.stderr = out, err
    code, crash = EXIT_OK, None
    try:
        code = _dispatch(argv)
        return code
    except SystemExit as exc:  # argparse usage errors, --help, --version
        code = exc.code if isinstance(exc.code, int) else int(exc.code is not None)
        raise
    except Exception:
        code, crash = 1, traceback.format_exc()
        raise
```

```
    finally:
        sys.stdout, sys.stderr = out.stream, err.stream
        if code:
            _log(argv, code, out, err, crash)


if __name__ == "__main__":  # pragma: no cover
    sys.exit(main())
```

The CLI tests check the contract, not the implementation: one stdout line, real PDF bytes, the black-text rule, each exit code, and blocked remote resources. `tests/test_cli.py`:

```
import io
import json

import pytest

from rupa.cli import EXIT_INPUT, EXIT_OK, EXIT_RENDER, EXIT_USAGE, main

DOC = "---\ntitle: T\n---\n\n## A\n\nBody text.\n"


def run(monkeypatch, capsys, argv, stdin=DOC):
    raw = stdin if isinstance(stdin, bytes) else stdin.encode()
    monkeypatch.setattr("sys.stdin", io.TextIOWrapper(io.BytesIO(raw)))
    code = main(argv)
    out, err = capsys.readouterr()
    return code, out, err


def test_html_from_stdin(tmp_path, monkeypatch, capsys):
    target = tmp_path / "out.html"
    code, out, _ = run(monkeypatch, capsys, ["-o", str(target)])
    assert code == EXIT_OK
    assert target.read_text().startswith("<!doctype html>")
    # stdout is one status line, never the document
    assert out.count("\n") == 1 and out.startswith("rupa: wrote")


def test_pdf_reports_pages(tmp_path, monkeypatch, capsys):
    target = tmp_path / "out.pdf"
    code, out, _ = run(monkeypatch, capsys, ["-o", str(target), "--json"])
    status = json.loads(out)
    assert code == EXIT_OK
    assert status["ok"] and status["format"] == "pdf" and status["pages"] ⩾ 1
    assert target.read_bytes()[:5] == b"%PDF-"
```

```
def test_pdf_text_is_forced_black(tmp_path):
    from rupa.document import parse
    from rupa.render import to_html

    html = to_html(parse(DOC), target="pdf")
    assert "color: #000000 !important" in html


def test_unknown_extension_is_usage_error(tmp_path, monkeypatch, capsys):
    code, _, err = run(monkeypatch, capsys, ["-o", str(tmp_path / "out.docx")])
    assert code == EXIT_USAGE and "use --to" in err


def test_bad_input_exit_code(tmp_path, monkeypatch, capsys):
    code, _, err = run(monkeypatch, capsys, ["-o", str(tmp_path / "o.html")], stdin="")
    assert code == EXIT_INPUT and "empty" in err


def test_unknown_theme_exit_code(tmp_path, monkeypatch, capsys):
    code, _, err = run(
        monkeypatch, capsys, ["-o", str(tmp_path / "o.html"), "--theme", "nope"]
    )
    assert code == EXIT_RENDER and "unknown theme" in err


@pytest.mark.parametrize("ext", ["pdf", "html"])
def test_remote_images_are_rejected(tmp_path, monkeypatch, capsys, ext):
    md = "## x\n\n![remote](https://example.com/a.png?d=secret)\n"
    out = str(tmp_path / f"o.{ext}")
    code, _, err = run(monkeypatch, capsys, ["-o", out], stdin=md)
    assert code == EXIT_INPUT and "example.com" in err


def _image_doc(tmp_path, src):
    from PIL import Image

    (tmp_path / "doc").mkdir()
    Image.new("RGB", (40, 20), "red").save(tmp_path / "doc" / "fig.png")
    Image.new("RGB", (40, 20)).save(tmp_path / "secret.png")
    md = tmp_path / "doc" / "t.md"
    md.write_text(f"## x\n\n![f]({src})\n")
    return str(md)


def test_relative_image_next_to_the_markdown_is_embedded(tmp_path, capsys):
    import pdfplumber
```

```
    target = tmp_path / "out" / "o.pdf"
    assert main([_image_doc(tmp_path, "fig.png"), "-o", str(target)]) == EXIT_OK
    with pdfplumber.open(target) as pdf:
        assert sum(len(p.images) for p in pdf.pages) == 1


def test_html_embeds_images_so_the_file_stands_alone(tmp_path, capsys):
    target = tmp_path / "out" / "o.html"
    assert main([_image_doc(tmp_path, "fig.png"), "-o", str(target)]) == EXIT_OK
    html = target.read_text()
    assert '<img src="' in html and "fig.png" not in html


@pytest.mark.parametrize("ext", ["pdf", "html"])
@pytest.mark.parametrize("src", ["missing.png", "../secret.png"])
def test_missing_or_outside_image_fails_the_render(tmp_path, capsys, src, ext):
    code = main([_image_doc(tmp_path, src), "-o", str(tmp_path / f"o.{ext}")])
    assert code == EXIT_RENDER and "resource" in capsys.readouterr().err


def test_to_flag_overrides_extension(tmp_path, monkeypatch, capsys):
    target = tmp_path / "report.out"
    code, _, _ = run(monkeypatch, capsys, ["-o", str(target), "--to", "pdf"])
    assert code == EXIT_OK and target.read_bytes()[:5] == b"%PDF-"


def test_any_weasyprint_failure_is_a_render_error(tmp_path, monkeypatch, capsys):
    def boom(self, *a, **kw):
        raise RuntimeError("layout exploded")

    monkeypatch.setattr("weasyprint.HTML.render", boom)
    code, _, err = run(monkeypatch, capsys, ["-o", str(tmp_path / "o.pdf")])
    assert code == EXIT_RENDER and "layout exploded" in err


def test_verify_on_a_non_pdf_is_bad_input(tmp_path, capsys):
    bad = tmp_path / "notes.pdf"
    bad.write_text("not a pdf")
    code = main(["verify", str(bad)])
    _, err = capsys.readouterr()
    assert code == EXIT_INPUT and err.startswith("rupa: error: cannot read PDF")
    assert err.count("\n") == 1


def test_verify_bad_wants_value_is_bad_input(tmp_path, capsys):
    pdf = tmp_path / "x.pdf"
    pdf.write_bytes(b"%PDF-1.4")
```

```
    wants = tmp_path / "wants.yaml"
    wants.write_text('text_color: "#GGG"\n')
    code = main(["verify", str(pdf), "--wants", str(wants)])
    assert code == EXIT_INPUT and "text_color" in capsys.readouterr().err


def test_verify_sample_flag_overrides_wants(tmp_path, monkeypatch, capsys):
    pdf = tmp_path / "o.pdf"
    run(monkeypatch, capsys, ["-o", str(pdf)])
    code = main(["verify", str(pdf), "--sample", "flagged", "--json"])
    report = json.loads(capsys.readouterr().out)
    assert code == EXIT_OK and report["ok"] and report["sheets"] == []


LONG = "---\ntitle: T\npages: {budget}\n---\n\n" + "\n\n".join(
    f"## S{i}\n\n" + "Owning identity, detection and response in the cloud. " * 30
    for i in range(6)
)


def test_page_budget_fits_by_tightening(tmp_path, monkeypatch, capsys):
    target = tmp_path / "o.pdf"
    code, out, _ = run(
        monkeypatch, capsys, ["-o", str(target), "--json"], stdin=LONG.format(budget=3)
    )
    status = json.loads(out)
    assert code == EXIT_OK and status["ok"]
    assert status["pages"] <= 3 and status["fit"] and status["cut_lines"] == 0


def test_page_budget_over_says_how_much_to_cut(tmp_path, monkeypatch, capsys):
    target = tmp_path / "o.pdf"
    code, out, _ = run(
        monkeypatch, capsys, ["-o", str(target)], stdin=LONG.format(budget=1)
    )
    assert code == 5 and target.exists()  # the file is still written
    assert "over the 1-page budget" in out and "lines" in out


def test_pages_flag_overrides_frontmatter(tmp_path, monkeypatch, capsys):
    target = tmp_path / "o.pdf"
    code, out, _ = run(
        monkeypatch,
        capsys,
        ["-o", str(target), "--pages", "1", "--json"],
        stdin=LONG.format(budget=9),
    )
    assert code == 5 and json.loads(out)["budget"] == 1
```

```
def test_no_budget_leaves_render_alone(tmp_path, monkeypatch, capsys):
    target = tmp_path / "o.pdf"
    code, out, _ = run(monkeypatch, capsys, ["-o", str(target), "--json"])
    assert code == EXIT_OK and "fit" not in json.loads(out)


def test_cut_estimate_counts_lines_of_code():
    # A JSON block has one or two words a line, so a word count of the
    # overflow says far too little. The estimate must cover the lines.
    import io

    import pdfplumber

    from rupa.document import parse
    from rupa.render import fit_pdf

    code = "\n".join(f'  "key{i}": "value",' for i in range(150))
    fit = fit_pdf(parse(f"# T\n\n```json\n{{\n{code}\n}}\n```\n"), budget=1)
    with pdfplumber.open(io.BytesIO(fit.data)) as pdf:
        lines_past = sum(
            len((p.extract_text() or "").splitlines()) - 1 for p in pdf.pages[1:]
        )  # less the footer line
    assert lines_past ⩽ fit.cut_lines ⩽ lines_past * 1.3


def test_pdf_embeds_bundled_fonts():
    # Fonts ship with Rupa, so the PDF doesn't depend on what is installed.
    import io

    import pdfplumber

    from rupa.document import parse
    from rupa.render import to_pdf

    data, _ = to_pdf(parse("# T\n\nBody **bold** and `code`.\n"))
    with pdfplumber.open(io.BytesIO(data)) as pdf:
        fonts = {c["fontname"].split("+")[-1] for c in pdf.pages[0].chars}
    assert any(f.startswith("Inter") for f in fonts)
    assert any(f.startswith("JetBrains") for f in fonts)


def _entries(path):
    return [json.loads(line) for line in path.read_text().splitlines()]


def test_success_is_not_logged(tmp_path, monkeypatch, capsys, _rupa_log):
```

```
    code, _, _ = run(monkeypatch, capsys, ["-o", str(tmp_path / "o.html")])
    assert code == EXIT_OK and not _rupa_log.exists()


def test_failure_is_logged(tmp_path, monkeypatch, capsys, _rupa_log):
    argv = ["-o", str(tmp_path / "o.html")]
    code, out, err = run(monkeypatch, capsys, argv, stdin="")
    (entry,) = _entries(_rupa_log)
    assert code == EXIT_INPUT and entry["exit"] == EXIT_INPUT
    assert entry["argv"] == argv and "empty" in entry["stderr"]
    # the contract is unchanged: the user still sees the same output
    assert "empty" in err and out == ""


def test_usage_error_is_logged(tmp_path, monkeypatch, capsys, _rupa_log):
    with pytest.raises(SystemExit):
        run(monkeypatch, capsys, ["-o", str(tmp_path / "o.html"), "--no-such-flag"])
    (entry,) = _entries(_rupa_log)
    assert entry["exit"] == EXIT_USAGE and "--no-such-flag" in entry["stderr"]


def test_crash_is_logged_with_traceback(tmp_path, monkeypatch, capsys, _rupa_log):
    def boom(*a, **k):
        raise RuntimeError("kaboom")

    monkeypatch.setattr("rupa.cli.to_html", boom)
    with pytest.raises(RuntimeError):
        run(monkeypatch, capsys, ["-o", str(tmp_path / "o.html")])
    (entry,) = _entries(_rupa_log)
    assert entry["exit"] == 1 and "kaboom" in entry["traceback"]


def test_log_off(tmp_path, monkeypatch, capsys, _rupa_log):
    monkeypatch.setenv("RUPA_LOG", "off")
    run(monkeypatch, capsys, ["-o", str(tmp_path / "o.html")], stdin="")
    assert not _rupa_log.exists()


def test_to_does_not_replace_a_file_that_isnt_html_or_pdf(
    tmp_path, monkeypatch, capsys
):
    target = tmp_path / "notes.txt"
    target.write_text("keep me")
    code, _, err = run(monkeypatch, capsys, ["-o", str(target), "--to", "html"])
    assert code == EXIT_USAGE and "--force" in err
    assert target.read_text() == "keep me"
    code, _, _ = run(
        monkeypatch, capsys, ["-o", str(target), "--to", "html", "--force"]
```

```
    )
    assert code == EXIT_OK and target.read_text().startswith("<!doctype html>")


def test_existing_html_and_pdf_are_still_replaced(tmp_path, monkeypatch, capsys):
    for name in ("o.html", "o.pdf"):
        target = tmp_path / name
        target.write_text("old")
        code, _, _ = run(monkeypatch, capsys, ["-o", str(target)])
        assert code == EXIT_OK and target.read_bytes() != b"old"


def _doc_with(tmp_path, name, data, src=None):
    (tmp_path / "doc").mkdir(exist_ok=True)
    (tmp_path / "doc" / name).write_bytes(data)
    md = tmp_path / "doc" / "t.md"
    md.write_text(f"## x\n\n![f]({src or name})\n")
    return str(md)


@pytest.mark.parametrize("ext", ["pdf", "html"])
@pytest.mark.parametrize("name", [".env", "notes.txt", "fake.png", ".hidden/fig.png"])
def test_files_that_arent_real_visible_images_are_refused(tmp_path, capsys, ext, name):
    from PIL import Image

    secret = b"API_KEY=sk-not-for-the-page"
    if name.startswith(".hidden"):
        (tmp_path / "doc" / ".hidden").mkdir(parents=True)
        Image.new("RGB", (4, 4)).save(tmp_path / "doc" / name)
        md = _doc_with(tmp_path, "unused.txt", b"", src=name)
    else:
        md = _doc_with(tmp_path, name, secret)
    target = tmp_path / f"o.{ext}"
    assert main([md, "-o", str(target)]) == EXIT_RENDER
    err = capsys.readouterr().err
    assert "blocked" in err or "cannot be decoded" in err
    assert not target.exists()


@pytest.mark.parametrize("ext", ["pdf", "html"])
def test_an_image_that_wont_decode_fails_the_render(tmp_path, capsys, ext):
    from PIL import Image

    buf = io.BytesIO()
    Image.effect_noise((200, 200), 64).save(buf, "PNG")
    data = buf.getvalue()
    md = _doc_with(tmp_path, "cut.png", data[: len(data) // 2])  # truncated
    code = main([md, "-o", str(tmp_path / f"o.{ext}")])
```

```
    assert code == EXIT_RENDER and "cut.png" in capsys.readouterr().err


@pytest.mark.parametrize("ext", ["pdf", "html"])
def test_svg_images_still_render(tmp_path, capsys, ext):
    svg = b'<svg xmlns="http://www.w3.org/2000/svg" width="40" height="20"><rect width="40"
height="20" fill="red"/></svg>'
    target = tmp_path / f"o.{ext}"
    assert main([_doc_with(tmp_path, "fig.svg", svg), "-o", str(target)]) == EXIT_OK
    if ext == "html":
        assert "" in target.read_text()


def test_non_utf8_stdin_is_bad_input_without_a_utf8_locale(
    tmp_path, monkeypatch, capsys
):
    # Under the C locale Python decodes stdin with surrogateescape, so bad
    # bytes used to slip through and crash the render (exit 1 or 4).
    stdin = io.TextIOWrapper(
        io.BytesIO(b"## caf\xe9\n"), encoding="utf-8", errors="surrogateescape"
    )
    monkeypatch.setattr("sys.stdin", stdin)
    for ext in ("html", "pdf"):
        stdin.seek(0)
        assert main(["-o", str(tmp_path / f"o.{ext}")]) == EXIT_INPUT
        assert "utf-8" in capsys.readouterr().err


@pytest.mark.parametrize(
    "stdin, needle",
    [
        ("---\ntitle: [a, b\nfoo: : :\n---\n# x\n", "bad frontmatter: line 2"),
        ("---\ntheme: " + "x" * 300 + "\n---\n# x\n", "unknown theme"),
        ("---\n" + "k" * 300 + ": 1\n---\n# x\n", "unknown frontmatter keys"),
        ("![a](http://" + "x" * 300 + ".png)\n", "must be a relative path"),
    ],
)
def test_errors_are_one_short_line(tmp_path, monkeypatch, capsys, stdin, needle):
    code, _, err = run(monkeypatch, capsys, ["-o", str(tmp_path / "o.pdf")], stdin)
    assert code != EXIT_OK and needle in err
    assert err.count("\n") == 1 and len(err) < 200


def test_bad_wants_yaml_is_one_line(tmp_path, monkeypatch, capsys):
    pdf = tmp_path / "x.pdf"
    run(monkeypatch, capsys, ["-o", str(pdf)])
    wants = tmp_path / "wants.yaml"
    wants.write_text("checks: [a, b\nfoo: : :\n")
```

```
    code = main(["verify", str(pdf), "--wants", str(wants)])
    err = capsys.readouterr().err
    assert code == EXIT_INPUT and "bad wants file: line 2" in err
    assert err.count("\n") == 1


def test_verify_missing_pdf_is_bad_input(tmp_path, capsys):
    assert main(["verify", str(tmp_path / "nope.pdf")]) == EXIT_INPUT
    assert "no such file" in capsys.readouterr().err


def test_file_named_verify_renders_with_dot_slash(tmp_path, monkeypatch, capsys):
    monkeypatch.chdir(tmp_path)
    (tmp_path / "verify").write_text(DOC)
    code = main(["./verify", "-o", "out.html"])
    capsys.readouterr()
    assert code == EXIT_OK and (tmp_path / "out.html").is_file()
```

Checkpoint: `uv run pytest -q` shows 16 passed.

Step 5: Install it as a command

```
uv tool install .
echo "# Hello" | rupa -o hello.pdf
```

Expected output:

```
rupa: wrote hello.pdf (pdf, 1 page, …)
```

Try the failure paths too, since Claude will hit them:

```
echo "" | rupa -o x.html          # exit 3: empty input
echo "# x" | rupa -o x.docx       # exit 2: cannot infer format
echo "# x" | rupa -o x.html --theme nope   # exit 4: unknown theme
```

After changing the code, reinstall with `uv tool install --reinstall .`.

Step 6: Continuous integration

CI installs the PDF system libraries, runs the tests, and does a smoke render. `.github/workflows/ci.yml`:

```yaml
name: ci
on: [push, pull_request]
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: System libraries for WeasyPrint
        run: |
          sudo apt-get update
          sudo apt-get install -y libpango-1.0-0 libpangoft2-1.0-0 libharfbuzz-subset0
      - uses: astral-sh/setup-uv@v6
      - run: uv sync
      - run: uvx ruff check src tests
      - run: uvx ruff format --check src tests
      - run: uv run pytest -q
        env:
          RUPA_GOLDEN_FAILURES: out/golden-failures
      - name: Upload golden-render failures
        if: failure()
        uses: actions/upload-artifact@v4
        with:
          name: golden-failures
          path: out/golden-failures/
          if-no-files-found: ignore
      - name: Smoke render
        run: |
          printf -- '---\ntitle: CI\n---\n\n## ok\n' | uv run rupa -o out/ci.pdf
          uv run rupa verify out/ci.pdf
```

Checkpoint: push to GitHub and confirm the workflow is green.

Step 7: The Claude Code plugin

Create the three plugin files, starting with `plugin/.claude-plugin/plugin.json`:

```
{
  "name": "rupa",
  "displayName": "Rupa",
  "version": "0.1.0",
  "description": "Write Markdown, let Rupa render the HTML or PDF. Keeps markup out of
Claude's output tokens.",
  "author": { "name": "Vajra Sarpa", "url": "https://vajrasarpa.com" },
  "license": "MIT",
  "keywords": ["render", "pdf", "html", "markdown"]
}
```

`plugin/skills/render/SKILL.md`. The `description` line is the only part loaded every session, so it carries the whole instruction in one sentence:

```
---
name: render
description: Use when the user wants an HTML page or PDF document. Write Markdown, render it
with the rupa CLI, then verify the PDF. Never hand-write HTML, CSS, or PDF code.
---

# Rendering with Rupa

## Render

1. Write the content as Markdown. Optional frontmatter keys: `title`,
   `subtitle`, `author`, `date`, `theme`, `toc` (true/false), `pages`.
   Themes: `default`, `cyan` (dark navy page on screen, black-on-white
   PDF), `cyan-dark` (dark PDF too). Use the one the user names; otherwise
   leave `theme` out. In `cyan`, a `**bold**` phrase in a heading is the
   cyan key phrase.
   No other keys. If the user asks for a page count, set `pages: N`: Rupa
   then fits the PDF to N pages itself (drops the contents list, tightens
   type). Plan about 30 lines per page: a line of prose is about 15 words,
   and each code line, table row, list item and heading is one line.
2. Use `##` for sections and `###` for subsections. Use `#` parts only in
   a long document (about 8 pages or more): each part starts a new page.
   Fenced code blocks take a language tag. Images are relative paths to
   PNG, JPEG, GIF, WebP or SVG files next to the Markdown file (or
   `data:image` URIs), never URLs.
   A PDF table fits about 11 short columns (Rupa sets 9 or more in smaller
   type). For more, or for long cell text, write two tables that each
```

```
   repeat the first column.
3. Render with one command, using the extension the user asked for:

   ```bash
   rupa draft.md -o out/report.pdf
   ```

4. Read only the one-line status. Do not open or print the output file.
5. On a non-zero exit, fix the Markdown: `3` bad input and `4` render
   failure explain themselves on stderr; `5` means the PDF is still over
   `pages` and the status line says about how many lines to cut, counted
   at the tightest type (about 18 words of prose a line). Cut that many
   by rewriting the whole Markdown file with one Write, not a series of
   Edits, then render again. Do not measure pages yourself.

Verify (PDF)

1. Run `rupa verify out/report.pdf` (add `--wants wants.yaml` if the project
   has one). It prints one line and writes `verify.json` plus sheet images.
2. If the status line reports no sheets, the PDF passed: skip to step 6.
   Do not render or open page images yourself.
   Otherwise read `verify.json`, then open each sheet image listed under `sheets`.
   Each sheet shows the theme's reference page on the left and the rendered
   page on the right.
3. Judge every sheet against each item in `rubric`. Record one line per
   sheet: `p<N> <kind>: PASS` or `p<N> <kind>: FAIL <rubric item> - <what>`.
4. Fix failures in the Markdown only: shorten a long code line, move or
   trim content around an awkward break. If `overflow` names a table, split
   it into two tables that each repeat the first column. Shortening cell
   text rarely closes the gap. Make each round's fixes with one Write of
   the whole file, then render and verify again. Stop after two fix rounds.
5. If a failure can only be fixed in the theme (CSS), do not edit CSS.
   Report it to the user as a theme issue.
6. Tell the user the final check line and your per-sheet verdicts in a few
   lines.

Do not add styling to the Markdown. A different look is a theme change.
```

`plugin/hooks/hooks.json`. It prints a warning at session start only if the CLI is missing:

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
```

```
            "type": "command",
            "command": "command -v rupa >/dev/null || echo 'rupa CLI not on PATH: run uv tool
install <path-to-rupa>'"
          }
        ]
      }
    ]
  }
}
```

Validate and load it:

```
claude plugin validate ./plugin
# ✓ Validation passed

claude --plugin-dir ./plugin
```

Plugin names can't start with `claude-` or `anthropic-`; `rupa` is fine.

Checkpoint: inside that session, ask Claude whether a skill named `rupa:render` is available. It should say yes. Then ask for a one-page PDF on any topic and check that it wrote Markdown and called `rupa`, rather than writing HTML.

Step 8: Verification

`src/rupa/verify.py` holds both tiers. It builds a small page model from the PDF (lines with size, weight and position; fill; table area; share of monospace text), runs the checks on that model, and writes the sheets. Keeping the checks on a plain model rather than on pdfplumber objects is what makes them unit-testable.

Two traps found while building it:

- WeasyPrint paints the page background as one large white rectangle. Left in, it makes every page look 100% full and turns every page into a "table". The code filters it out before measuring.
- The cover page ends in a forced page break after the contents, so it is often short by design. It is exempt from the fill checks, as is the last page.
- A reference page that spills onto a second page is a broken reference: the review compares against half a page. The visual review of this document caught exactly that after a sample edit, so `rupa refs` now refuses a sample whose page for any type runs over.

```
"""Verify a rendered PDF against a wants file.

Two tiers:
  1. Deterministic checks, run here in code: color, size, page count, empty
     pages, stranded headings, overflow. Free and repeatable.
  2. Evidence for a vision review: a side-by-side sheet (reference page |
     rendered page) for every flagged page, and with sample: per-type one
     per page type too. Rupa never calls
     a model; Claude reads the sheets and judges them against the rubric.
"""

from __future__ import annotations

import json
import re
from dataclasses import asdict, dataclass, field
from importlib import resources
from pathlib import Path

import yaml

from .document import clip, yaml_problem

PT_PER_IN = 72
PAGE_SIZES = {"letter": (612, 792), "a4": (595, 842)}
# Body box from print.css (@page margins). Footer sits below BODY_BOTTOM.
MARGIN_TOP, MARGIN_SIDE, MARGIN_BOTTOM = 0.85, 0.8, 0.9
FOOTER_CUTOFF = 0.8  # inches from the bottom edge; lines below are footer
PART_HEADING_PT = 18  # an H1 "part" title is at least this size
HEADING_PT = 11.5  # bold, non-mono text at least this size is a heading

ALL_CHECKS = (
    "text-color",
    "page-size",
    "max-pages",
    "blank-page",
    "near-empty",
    "sparse",
    "heading-at-bottom",
    "overflow",
)
DEFAULT_RUBRIC = [
    "hierarchy: titles, headings and body read in the same order and weight as the reference",
    "spacing: margins, gaps and alignment match the reference; nothing crowded or floating",
    "overflow: no text, table or code is clipped, cut off or running into the margin",
    "page balance: no awkward breaks (a lone row, a stranded heading, a near-empty page)",
    "style: colors, rules and boxes match the reference; text is solid black",
```

```
]
SAMPLE_MODES = ("per-type", "flagged", "all", "none")
DEFAULT_WANTS = {
    "theme": "default",
    "page_size": "letter",
    "max_pages": 100,
    "text_color": None,  # None: the theme's colours (THEME_TEXT_COLORS in render.py)
    "sparse_fill": 0.4,
    "near_empty_fill": 0.2,
    "checks": list(ALL_CHECKS),
    "vision": {
        "sample": "flagged",
        "max_sheets": 8,
        "resolution": 90,
        "rubric": DEFAULT_RUBRIC,
    },
}
EXIT_CHECKS_FAILED = 5


class WantsError(ValueError):
    """Raised for an unreadable or invalid wants file."""


class PdfError(ValueError):
    """Raised when the file given to verify cannot be read as a PDF."""


# ---------------------------------------------------------------- wants


def load_wants(path: str | None) -> dict:
    wants = json.loads(json.dumps(DEFAULT_WANTS))  # deep copy
    if not path:
        return wants
    try:
        given = yaml.safe_load(Path(path).read_text(encoding="utf-8")) or {}
    except OSError as exc:
        raise WantsError(f"cannot read wants file: {exc}") from exc
    except yaml.YAMLError as exc:
        raise WantsError(f"bad wants file: {yaml_problem(exc)}") from exc
    if not isinstance(given, dict):
        raise WantsError("wants file must be a YAML mapping")
    unknown = set(given) - set(DEFAULT_WANTS)
    if unknown:
        raise WantsError(f"unknown wants keys: {clip(sorted(map(str, unknown)))}")
    vision = given.pop("vision", {}) or {}
    if not isinstance(vision, dict):
```

```
        raise WantsError("vision must be a YAML mapping")
    unknown_v = set(vision) - set(DEFAULT_WANTS["vision"])
    if unknown_v:
        raise WantsError(f"unknown vision keys: {clip(sorted(map(str, unknown_v)))}")
    wants.update(given)
    wants["vision"].update(vision)
    _validate(wants)
    return wants


def _is_int(value) -> bool:
    return isinstance(value, int) and not isinstance(value, bool)


def _is_fraction(value) -> bool:
    return (
        isinstance(value, (int, float))
        and not isinstance(value, bool)
        and (0 ⩽ value ⩽ 1)
    )


def _is_str_list(value) -> bool:
    return isinstance(value, list) and all(isinstance(v, str) for v in value)


def _validate(wants: dict) -> None:
    """Reject bad values here, so verify() never fails halfway with a traceback."""
    from .render import THEMES

    vision = wants["vision"]
    if wants["theme"] not in THEMES:
        raise WantsError(f"theme must be one of {list(THEMES)}")
    if wants["page_size"] not in PAGE_SIZES:
        raise WantsError(f"page_size must be one of {sorted(PAGE_SIZES)}")
    if not _is_int(wants["max_pages"]) or wants["max_pages"] < 1:
        raise WantsError("max_pages must be a whole number of at least 1")
    color = wants["text_color"]
    if color is not None and not (
        isinstance(color, str) and re.fullmatch(r"#[0-9a-fA-F]{6}", color)
    ):
        raise WantsError(f"text_color must be #RRGGBB, got {color!r}")
    for key in ("sparse_fill", "near_empty_fill"):
        if not _is_fraction(wants[key]):
            raise WantsError(f"{key} must be a number from 0 to 1")
    if not _is_str_list(wants["checks"]):
        raise WantsError("checks must be a list of check names")
    bad = set(wants["checks"]) - set(ALL_CHECKS)
```

```
    if bad:
        raise WantsError(f"unknown checks: {sorted(bad)}")
    if vision["sample"] not in SAMPLE_MODES:
        raise WantsError(f"vision.sample must be one of {', '.join(SAMPLE_MODES)}")
    if not _is_int(vision["max_sheets"]) or vision["max_sheets"] < 0:
        raise WantsError("vision.max_sheets must be a whole number of at least 0")
    if not _is_int(vision["resolution"]) or not 30 ⩽ vision["resolution"] ⩽ 300:
        raise WantsError("vision.resolution must be a whole number from 30 to 300")
    if not _is_str_list(vision["rubric"]):
        raise WantsError("vision.rubric must be a list of strings")


# ---------------------------------------------------------------- page model


@dataclass
class Line:
    top: float
    bottom: float
    x0: float
    x1: float
    size: float
    mono: bool
    bold: bool
    text: str


@dataclass
class PageInfo:
    number: int
    width: float
    height: float
    lines: list[Line] = field(default_factory=list)  # body lines only, no footer
    colors: list[tuple] = field(
        default_factory=list
    )  # one entry per distinct color seen
    off_colors: int = 0  # number of chars not matching the wanted color
    content_bottom: float = 0.0
    table_frac: float = 0.0
    tables: list[tuple] = field(default_factory=list)  # (x0, top, x1, bottom)
    image_bottom: float = 0.0  # lowest image edge in the body; images have no text
    kind: str = "prose"

    @property
    def body_top(self) -> float:
        return MARGIN_TOP * PT_PER_IN

    @property
```

```
    def body_bottom(self) -> float:
        return self.height - MARGIN_BOTTOM * PT_PER_IN

    @property
    def fill(self) -> float:
        if not self.lines:
            return 0.0
        span = self.body_bottom - self.body_top
        if span <= 0:  # a page too small to have a body box
            return 0.0
        return max(0.0, min(1.0, (self.content_bottom - self.body_top) / span))

    @property
    def mono_frac(self) -> float:
        chars = sum(len(ln.text) for ln in self.lines) or 1
        return sum(len(ln.text) for ln in self.lines if ln.mono) / chars

    @property
    def starts_part(self) -> bool:
        # The first or second line: a theme may set a small label above a part title.
        return any(ln.size >= PART_HEADING_PT for ln in self.lines[:2])


def _hex_to_rgb(value: str) -> tuple[float, float, float]:
    v = value.lstrip("#")
    if len(v) != 6:
        raise WantsError(f"text_color must be #RRGGBB, got {value!r}")
    return tuple(int(v[i : i + 2], 16) / 255 for i in (0, 2, 4))


def _to_rgb(color) -> tuple[float, float, float] | None:
    """Normalize a pdfplumber color (gray, RGB or CMYK tuple) to RGB."""
    if color is None:
        return None
    c = tuple(color) if isinstance(color, (list, tuple)) else (color,)
    if len(c) == 1:
        return (c[0], c[0], c[0])
    if len(c) == 3:
        return c
    if len(c) == 4:
        cy, m, y, k = c
        return ((1 - cy) * (1 - k), (1 - m) * (1 - k), (1 - y) * (1 - k))
    return None


def _same_color(a, b, tol: float = 0.01) -> bool:
    return a is not None and all(abs(x - y) <= tol for x, y in zip(a, b))
```

```
def _text_colors(value) -> tuple[str, ...]:
    """A wants text_color (one colour, several, or None) as a tuple of colours."""
    if value is None:
        return ("#000000",)
    return (value,) if isinstance(value, str) else tuple(value)


def read_pages(pdf_path: str | Path, wanted_color="#000000") -> list[PageInfo]:
    """wanted_color: one #RRGGBB or several; any other text colour counts as off."""
    import pdfplumber

    want = [_hex_to_rgb(c) for c in _text_colors(wanted_color)]
    try:
        return _read_pages(pdfplumber, pdf_path, want)
    except Exception as exc:  # pdfminer raises many types on a damaged file
        raise PdfError(f"cannot read PDF {pdf_path}: {exc}") from exc


def _read_pages(pdfplumber, pdf_path, want) -> list[PageInfo]:
    pages: list[PageInfo] = []
    with pdfplumber.open(pdf_path) as pdf:
        for i, page in enumerate(pdf.pages, start=1):
            info = PageInfo(
                number=i, width=float(page.width), height=float(page.height)
            )
            seen: set = set()
            for ch in page.chars:
                rgb = _to_rgb(ch.get("non_stroking_color"))
                seen.add(rgb)
                if not any(_same_color(rgb, w) for w in want):
                    info.off_colors += 1
            info.colors = sorted(c for c in seen if c is not None)
            footer_y = info.height - FOOTER_CUTOFF * PT_PER_IN
            for tl in page.extract_text_lines(return_chars=True):
                if tl["top"] >= footer_y or not tl["chars"]:
                    continue
                chars = tl["chars"]
                fonts = [c.get("fontname", "") for c in chars]
                info.lines.append(
                    Line(
                        top=tl["top"],
                        bottom=tl["bottom"],
                        x0=tl["x0"],
                        x1=tl["x1"],
                        size=max(c["size"] for c in chars),
                        mono=sum("Mono" in f for f in fonts) > len(fonts) / 2,
                        bold=sum(
```

```
                            any(w in f for w in ("Bold", "SemiBold", "Black"))
                            for f in fonts
                        )
                        > len(fonts) / 2,
                        text=tl["text"],
                    )
                )
            body_h = info.body_bottom - info.body_top
            # The page background is painted as one big white rect; ignore it and
            # anything else that spans nearly the whole body.
            rects = [
                r
                for r in page.rects
                if r["bottom"] < footer_y and r["height"] < 0.95 * body_h
            ]
            line_bottoms = [ln.bottom for ln in info.lines]
            info.image_bottom = max(
                (im["bottom"] for im in page.images if im["bottom"] < footer_y),
                default=0.0,
            )
            info.content_bottom = max(
                [r["bottom"] for r in rects] + line_bottoms + [info.image_bottom],
                default=0.0,
            )
            body_area = (info.width - 2 * MARGIN_SIDE * PT_PER_IN) * body_h
            big = 0.95 * body_h
            no_bg = page.filter(
                lambda o, big=big: (
                    not (o.get("object_type") == "rect" and o["height"] >= big)
                )
            )
            tables = [t for t in no_bg.find_tables() if len(t.columns) >= 2]
            info.tables = [t.bbox for t in tables]
            table_area = sum(
                (t.bbox[2] - t.bbox[0]) * (t.bbox[3] - t.bbox[1]) for t in tables
            )
            info.table_frac = min(1.0, table_area / body_area) if body_area else 0.0
            info.kind = classify(info)
            pages.append(info)
    return pages


def classify(p: PageInfo) -> str:
    if p.number == 1:
        return "cover"
    if p.mono_frac > 0.5:
        return "code"
    if p.table_frac > 0.3:
```

```
        return "table"
    return "prose"


# ---------------------------------------------------------------- checks


@dataclass
class CheckResult:
    id: str
    ok: bool
    pages: list[int] = field(default_factory=list)
    detail: str = ""


def run_checks(pages: list[PageInfo], wants: dict) -> list[CheckResult]:
    enabled = wants["checks"]
    results: list[CheckResult] = []
    last = len(pages)

    def add(cid: str, bad: list[int], detail: str) -> None:
        if cid in enabled:
            results.append(CheckResult(cid, not bad, bad, detail if bad else ""))

    add(
        "text-color",
        [p.number for p in pages if p.off_colors],
        f"text not {' or '.join(_text_colors(wants['text_color']))}",
    )
    w, h = PAGE_SIZES[wants["page_size"]]
    add(
        "page-size",
        [p.number for p in pages if abs(p.width - w) > 1 or abs(p.height - h) > 1],
        f"page is not {wants['page_size']}",
    )
    add(
        "max-pages",
        list(range(wants["max_pages"] + 1, last + 1)),
        f"{last} pages > max {wants['max_pages']}",
    )
    add("blank-page", [p.number for p in pages if not p.lines], "page has no text")

    near_empty = [
        p.number
        for p in pages
        # The cover (page 1) ends in a forced break and the last page ends the
        # document, so neither is judged on how full it is.
        if p.lines and p.number not in (1, last) and p.fill < wants["near_empty_fill"]
```

```
    ]
    add("near-empty", near_empty, f"under {wants['near_empty_fill']:.0%} full")

    sparse = []
    for idx, p in enumerate(pages):
        if not p.lines or p.number in (1, last) or p.number in near_empty:
            continue
        next_starts_part = idx + 1 < last and pages[idx + 1].starts_part
        if p.fill < wants["sparse_fill"] and not next_starts_part:
            sparse.append(p.number)
    add(
        "sparse",
        sparse,
        f"under {wants['sparse_fill']:.0%} full with no part break after",
    )

    def is_heading(ln: Line) -> bool:
        return ln.bold and not ln.mono and ln.size ⩾ HEADING_PT

    stranded = []
    # The last page is exempt: nothing follows its final line, so no page break
    # separated a heading from its content (this also covers a title-only doc).
    for idx, p in enumerate(pages[:-1]):
        if not p.lines or p.image_bottom > p.lines[-1].bottom:
            continue  # an image after the last line is the heading's content
        if is_heading(p.lines[-1]):
            stranded.append(p.number)
            continue
        # A heading and a one-line lead-in ("Run this:") whose block moved to
        # the next page. A complete one-line section before a new heading is fine.
        nxt = pages[idx + 1].lines
        if (
            len(p.lines) ⩾ 2
            and is_heading(head := p.lines[-2])
            and p.lines[-1].top - head.bottom < 2 * head.size  # directly under it
            and nxt
            and not is_heading(nxt[0])
        ):
            stranded.append(p.number)
    add(
        "heading-at-bottom",
        stranded,
        "heading is the last line, or a heading and one line end the page",
    )

    overflow, where = [], []
    for p in pages:
        left = MARGIN_SIDE * PT_PER_IN - 2
```

```
        right = p.width - MARGIN_SIDE * PT_PER_IN + 2
        bad = [ln for ln in p.lines if ln.x0 < left or ln.x1 > right]
        if bad:
            overflow.append(p.number)
            over = max(max(left - ln.x0, ln.x1 - right) for ln in bad) + 2
            where.append(f"p{p.number} {_overflow_kind(p, bad)} by {over:.0f}pt")
    add(
        "overflow",
        overflow,
        f"text extends past the side margins ({', '.join(where)})",
    )
    return results


def _overflow_kind(p: PageInfo, bad: list[Line]) -> str:
    """Name what runs over, so the fix is clear: a table, code, or text."""

    def in_table(ln: Line) -> bool:
        mid = (ln.top + ln.bottom) / 2
        return any(top <= mid <= bottom for _, top, _, bottom in p.tables)

    if any(in_table(ln) for ln in bad):
        return "table"
    return "code" if any(ln.mono for ln in bad) else "text"


# ---------------------------------------------------------------- sampling + sheets


def sample_pages(
    pages: list[PageInfo], results: list[CheckResult], wants: dict
) -> list[PageInfo]:
    """Flagged pages first, then one representative (the middle one) per page kind.

    "flagged" stops after the flagged pages, so a clean PDF gets no sheets.
    """
    mode = wants["vision"]["sample"]
    if mode == "none":
        return []
    if mode == "all":
        return pages[: wants["vision"]["max_sheets"]]
    flagged = sorted({n for r in results for n in r.pages if n <= len(pages)})
    chosen: list[int] = list(flagged)
    for kind in () if mode == "flagged" else ("cover", "prose", "table", "code"):
        of_kind = [p.number for p in pages if p.kind == kind]
        if of_kind and not any(n in of_kind for n in chosen):
            chosen.append(of_kind[len(of_kind) // 2])
    by_number = {p.number: p for p in pages}
```

```
    return [by_number[n] for n in chosen[: wants["vision"]["max_sheets"]]]


def reference_path(theme: str, kind: str):
    ref = resources.files("rupa") / "references" / theme / f"{kind}.png"
    return ref if ref.is_file() else None


def _label(img, text: str):
    from PIL import Image, ImageDraw, ImageFont

    strip = 28
    out = Image.new("RGB", (img.width, img.height + strip), "white")
    out.paste(img, (0, strip))
    draw = ImageDraw.Draw(out)
    try:
        font = ImageFont.load_default(size=16)
    except TypeError:  # Pillow < 10.1
        font = ImageFont.load_default()
    draw.text((8, 5), text, fill="black", font=font)
    return out


# A sheet's rendered page is at most this many pixels on its long side, so an
# oversized page can't exhaust memory. Letter at 300 dpi is 3300.
MAX_SHEET_PX = 3300


def render_page_png(pdf_path: str | Path, number: int, resolution: int):
    import pdfplumber

    with pdfplumber.open(pdf_path) as pdf:
        page = pdf.pages[number - 1]
        long_side = max(float(page.width), float(page.height))
        if min(float(page.width), float(page.height)) <= 0:
            raise PdfError(f"page {number} has no size ({page.width}x{page.height}pt)")
        resolution = min(resolution, MAX_SHEET_PX * PT_PER_IN / long_side)
        return page.to_image(resolution=resolution).original.convert("RGB")


def make_sheet(
    pdf_path, page: PageInfo, flags: list[str], theme: str, resolution: int, out: Path
) -> Path:
    from PIL import Image

    rendered = render_page_png(pdf_path, page.number, resolution)
    right = _label(
        rendered,
```

```
        f"PAGE {page.number} ({page.kind})"
        + (f"  FLAGGED: {', '.join(flags)}" if flags else ""),
    )
    ref = reference_path(theme, page.kind)
    if ref is None:
        sheet = right
    else:
        with resources.as_file(ref) as ref_file:
            ref_img = Image.open(ref_file).convert("RGB")
            ref_img = ref_img.resize(
                (
                    round(ref_img.width * rendered.height / ref_img.height),
                    rendered.height,
                )
            )
        left = _label(ref_img, f"REFERENCE: {page.kind} ({theme})")
        sheet = Image.new(
            "RGB", (left.width + right.width + 16, right.height), (148, 163, 184)
        )
        sheet.paste(left, (0, 0))
        sheet.paste(right, (left.width + 16, 0))
    path = out / f"sheet-p{page.number:03d}-{page.kind}.png"
    sheet.save(path, optimize=True)
    return path


# ---------------------------------------------------------------- entry point


@dataclass
class VerifyReport:
    pdf: str
    pages: int
    checks: list[CheckResult]
    sheets: list[dict]
    rubric: list[str]
    manifest: str = ""

    @property
    def ok(self) -> bool:
        return all(c.ok for c in self.checks)

    def summary(self) -> str:
        passed = sum(c.ok for c in self.checks)
        fails = [
            f"{c.id} p{','.join(map(str, c.pages[:6]))}"
            for c in self.checks
            if not c.ok
```

```
        ]
        line = f"verify: {passed}/{len(self.checks)} checks pass"
        if fails:
            line += f" (FAIL {'; '.join(fails)})"
        if self.sheets:
            n = len(self.sheets)
            line += (
                f"; {n} sheet{'s' if n != 1 else ''} for visual review"
                f" in {Path(self.manifest).parent}/"
            )
        return line


def pdf_theme(pdf_path: str | Path) -> str | None:
    """The theme Rupa rendered this PDF with (its keywords metadata), if known."""
    import pdfplumber

    from .render import THEMES

    try:
        with pdfplumber.open(pdf_path) as pdf:
            keywords = str(pdf.metadata.get("Keywords", ""))
    except Exception:  # noqa: BLE001 - an unreadable PDF is reported by read_pages
        return None
    m = re.search(r"rupa-theme:([\w-]+)", keywords)
    return m.group(1) if m and m.group(1) in THEMES else None


def verify(
    pdf_path: str | Path, wants: dict, out_dir: str | Path, theme: str | None = None
) -> VerifyReport:
    from .render import THEME_TEXT_COLORS

    # --theme, else the theme Rupa recorded in the PDF, else the wants file's.
    theme = theme or pdf_theme(pdf_path) or wants["theme"]
    if not wants["text_color"]:
        wants = {**wants, "text_color": THEME_TEXT_COLORS[theme]}
    out = Path(out_dir)
    out.mkdir(parents=True, exist_ok=True)
    for old in out.glob("sheet-p*.png"):
        old.unlink()
    pages = read_pages(pdf_path, wants["text_color"])
    if not pages:  # every check loops over pages, so none would all "pass"
        raise PdfError(f"{pdf_path} has no pages")
    results = run_checks(pages, wants)
    flags_by_page: dict[int, list[str]] = {}
    for r in results:
        for n in r.pages:
```

```
            flags_by_page.setdefault(n, []).append(r.id)
    sheets = []
    for p in sample_pages(pages, results, wants):
        flags = flags_by_page.get(p.number, [])
        try:
            path = make_sheet(
                pdf_path, p, flags, theme, wants["vision"]["resolution"], out
            )
        except PdfError:
            raise
        except Exception as exc:  # pdfium and Pillow raise many types
            raise PdfError(f"cannot make the sheet for page {p.number}: {exc}") from exc
        sheets.append(
            {"path": str(path), "page": p.number, "kind": p.kind, "flags": flags}
        )
    report = VerifyReport(
        pdf=str(pdf_path),
        pages=len(pages),
        checks=results,
        sheets=sheets,
        rubric=wants["vision"]["rubric"],
        manifest=str(out / "verify.json"),
    )
    Path(report.manifest).write_text(
        json.dumps(
            {
                "pdf": report.pdf,
                "pages": report.pages,
                "ok": report.ok,
                "checks": [asdict(c) for c in results],
                "sheets": sheets,
                "rubric": report.rubric,
            },
            indent=2,
        )
    )
    return report


# ---------------------------------------------------------------- references


def build_references(
    pdf_path: str | Path, out_dir: str | Path, resolution: int = 90
) -> dict[str, int]:
    """Save the most typical page of each kind in a sample render as <kind>.png.

    Raises ValueError if the sample is missing a page kind, or if a picked page
```

```
    spills onto the next page, since a reference must show one whole page.
    """
    pages = read_pages(pdf_path)
    best: dict[str, PageInfo] = {}
    score = {
        "cover": lambda p: 1,
        "code": lambda p: p.mono_frac * p.fill,
        "table": lambda p: p.table_frac * p.fill,
        "prose": lambda p: p.fill,
    }
    for p in pages:
        if p.kind not in best or score[p.kind](p) > score[p.kind](best[p.kind]):
            best[p.kind] = p
    missing = {"cover", "prose", "table", "code"} - set(best)
    if missing:
        raise ValueError(f"sample has no {sorted(missing)} page; extend it")
    for kind, p in best.items():
        nxt = p.number  # pages are 1-based, so this indexes the following page
        if kind != "cover" and nxt < len(pages) and pages[nxt].kind == kind:
            raise ValueError(f"sample {kind} page p{p.number} spills over; trim it")
    out = Path(out_dir)
    out.mkdir(parents=True, exist_ok=True)
    for kind, p in best.items():
        render_page_png(pdf_path, p.number, resolution).save(
            out / f"{kind}.png", optimize=True
        )
    return {kind: p.number for kind, p in best.items()}
```

The sample document every theme's reference pages come from, `src/rupa/references/sample.md`. Each section is sized to fill one page of its type:

```
---
title: Reference Sample
subtitle: The page types every Rupa theme is compared against
author: Rupa
date: Reference
---

# Part One: Prose

## Body text

Rupa separates content from form. Claude writes Markdown, and the renderer applies every
visual decision in one place, the same way each time. This paragraph exists to show how body
text, spacing and rhythm look on a full page of prose, so a reviewer can compare a rendered
page against it. Rupa separates content from form. Claude writes Markdown, and the renderer
```

```
applies every visual decision in one place, the same way each time. This paragraph exists to
show how body text, spacing and rhythm look on a full page of prose, so a reviewer can compare
a rendered page against it.

Rupa separates content from form. Claude writes Markdown, and the renderer applies every
visual decision in one place, the same way each time. This paragraph exists to show how body
text, spacing and rhythm look on a full page of prose, so a reviewer can compare a rendered
page against it. Rupa separates content from form. Claude writes Markdown, and the renderer
applies every visual decision in one place, the same way each time. This paragraph exists to
show how body text, spacing and rhythm look on a full page of prose, so a reviewer can compare
a rendered page against it.

### A subsection

Rupa separates content from form. Claude writes Markdown, and the renderer applies every
visual decision in one place, the same way each time. This paragraph exists to show how body
text, spacing and rhythm look on a full page of prose, so a reviewer can compare a rendered
page against it. Rupa separates content from form. Claude writes Markdown, and the renderer
applies every visual decision in one place, the same way each time. This paragraph exists to
show how body text, spacing and rhythm look on a full page of prose, so a reviewer can compare
a rendered page against it.

- A list item with **bold** text and `inline code`.
- A second item that runs long enough to wrap onto a second line of the page.
- A third item.

> A blockquote, used for a note or caution in the body of the document.

# Part Two: Tables

## A table page

| Item | Component | Description | Estimate |
|---|---|---|---|
| Item 1 | Component 1 | A short description of what this row covers | 3 hours |
| Item 2 | Component 2 | A short description of what this row covers | 6 hours |
| Item 3 | Component 3 | A short description of what this row covers | 9 hours |
| Item 4 | Component 4 | A short description of what this row covers | 12 hours |
| Item 5 | Component 5 | A short description of what this row covers | 15 hours |
| Item 6 | Component 6 | A short description of what this row covers | 18 hours |
| Item 7 | Component 7 | A short description of what this row covers | 21 hours |
| Item 8 | Component 8 | A short description of what this row covers | 24 hours |
| Item 9 | Component 9 | A short description of what this row covers | 27 hours |
| Item 10 | Component 10 | A short description of what this row covers | 30 hours |
| Item 11 | Component 11 | A short description of what this row covers | 33 hours |
| Item 12 | Component 12 | A short description of what this row covers | 36 hours |
| Item 13 | Component 13 | A short description of what this row covers | 39 hours |
| Item 14 | Component 14 | A short description of what this row covers | 42 hours |
```

```
| Item 15 | Component 15 | A short description of what this row covers | 45 hours |
| Item 16 | Component 16 | A short description of what this row covers | 48 hours |
| Item 17 | Component 17 | A short description of what this row covers | 51 hours |
| Item 18 | Component 18 | A short description of what this row covers | 54 hours |

# Part Three: Code

## A code page

```python
def step_1(doc):
    """Stage 1 of the sample pipeline."""
    return transform(doc, stage=1)


def step_2(doc):
    """Stage 2 of the sample pipeline."""
    return transform(doc, stage=2)


def step_3(doc):
    """Stage 3 of the sample pipeline."""
    return transform(doc, stage=3)


def step_4(doc):
    """Stage 4 of the sample pipeline."""
    return transform(doc, stage=4)


def step_5(doc):
    """Stage 5 of the sample pipeline."""
    return transform(doc, stage=5)


def step_6(doc):
    """Stage 6 of the sample pipeline."""
    return transform(doc, stage=6)


def step_7(doc):
    """Stage 7 of the sample pipeline."""
    return transform(doc, stage=7)
```
```

Build the reference pages, and rebuild them whenever the theme changes:

```
uv run rupa refs
# rupa: wrote references for 'default' to .../references/default (code=p4, cover=p1, prose=p2,
table=p3)
```

The wants file with every default spelled out, `wants.example.yaml`. A test keeps it in sync with the code:

```
# What a finished PDF must satisfy. Every key is optional; omitted keys use
# the defaults shown here. Unknown keys are an error.
theme: default            # reference pages to compare against, when neither
                          # --theme nor the PDF (Rupa records it) names one
page_size: letter         # letter | a4
max_pages: 100
text_color: null          # every character must be this #RRGGBB; null uses the
                          # theme's colours (black for light themes)
sparse_fill: 0.4          # flag a page under 40% full, unless a part starts next
near_empty_fill: 0.2      # flag a page under 20% full, always
checks:                   # remove any you don't want
  - text-color
  - page-size
  - max-pages
  - blank-page
  - near-empty
  - sparse
  - heading-at-bottom
  - overflow
vision:
  sample: flagged         # flagged | per-type | all | none (per-type: flagged pages plus one
of each kind)
  max_sheets: 8           # cap on sheet images per run
  resolution: 90          # DPI for page images; ~1,500 x 1,000 px per sheet
  rubric:                 # what Claude judges on each sheet
    - "hierarchy: titles, headings and body read in the same order and weight as the
reference"
    - "spacing: margins, gaps and alignment match the reference; nothing crowded or floating"
    - "overflow: no text, table or code is clipped, cut off or running into the margin"
    - "page balance: no awkward breaks (a lone row, a stranded heading, a near-empty page)"
    - "style: colors, rules and boxes match the reference; text is solid black"
```

The tests, `tests/test_verify.py`. Most build synthetic pages, so each check is tested in isolation, and two run against a real render:

```
import json
import re

import pytest

from rupa.cli import EXIT_CHECKS, EXIT_INPUT, EXIT_OK, main
from rupa.document import parse
from rupa.render import to_pdf
from rupa.verify import (
    DEFAULT_WANTS,
    MARGIN_SIDE,
    PT_PER_IN,
    Line,
    PageInfo,
    WantsError,
    _to_rgb,
    load_wants,
    read_pages,
    run_checks,
    sample_pages,
    verify,
)

LETTER = (612.0, 792.0)


def body_line(top, size=10.5, bold=False, mono=False, x0=60.0, x1=500.0):
    return Line(top, top + size, x0, x1, size, mono, bold, "text")


def page(n, fill_to=700.0, lines=None, **kw):
    lines = lines if lines is not None else [body_line(70), body_line(fill_to - 10.5)]
    p = PageInfo(number=n, width=LETTER[0], height=LETTER[1], lines=lines, **kw)
    p.content_bottom = max((ln.bottom for ln in lines), default=0.0)
    return p


def failing(results):
    return {r.id: r.pages for r in results if not r.ok}


# ---- wants file
```

```
def test_defaults_without_a_file():
    assert load_wants(None) == DEFAULT_WANTS


def test_wants_file_overrides_and_merges_vision(tmp_path):
    f = tmp_path / "wants.yaml"
    f.write_text("max_pages: 3\nvision:\n  max_sheets: 2\n")
    wants = load_wants(str(f))
    assert wants["max_pages"] == 3
    assert wants["vision"]["max_sheets"] == 2
    assert wants["vision"]["sample"] == "flagged"  # untouched default survives


@pytest.mark.parametrize(
    "body, message",
    [
        ("colour: red\n", "unknown wants keys"),
        ("checks: [nope]\n", "unknown checks"),
        ("page_size: a5\n", "page_size"),
        ("vision:\n  model: big\n", "unknown vision keys"),
        ('text_color: "#GGG"\n', "text_color"),
        ('text_color: "#GGGGGG"\n', "text_color"),
        ('max_pages: "10"\n', "max_pages"),
        ("checks: null\n", "checks must be a list"),
        ("sparse_fill: 2\n", "sparse_fill"),
        ("theme: ../../etc\n", "theme must be one of"),
        ("vision: 5\n", "vision must be a YAML mapping"),
        ("vision:\n  resolution: 5000\n", "vision.resolution"),
    ],
)
def test_bad_wants_are_rejected(tmp_path, body, message):
    f = tmp_path / "wants.yaml"
    f.write_text(body)
    with pytest.raises(WantsError, match=message):
        load_wants(str(f))


# ---- colors


def test_color_normalization_handles_gray_rgb_and_cmyk():
    assert _to_rgb((0,)) == (0, 0, 0)
    assert _to_rgb((0.0, 0.0, 0.0)) == (0, 0, 0)
    assert _to_rgb((0, 0, 0, 1)) == (0, 0, 0)
    assert _to_rgb(None) is None
```

```
# ---- deterministic checks on synthetic pages


def test_clean_document_passes_everything():
    pages = [page(1), page(2), page(3)]
    assert failing(run_checks(pages, load_wants(None))) == {}


def test_off_color_text_fails():
    pages = [page(1), page(2, off_colors=12)]
    assert failing(run_checks(pages, load_wants(None))) == {"text-color": [2]}


def test_wrong_page_size_fails():
    pages = [page(1)]
    pages[0].width, pages[0].height = 595.0, 842.0  # A4
    assert "page-size" in failing(run_checks(pages, load_wants(None)))


def test_near_empty_page_fails_but_last_page_is_exempt():
    pages = [page(1), page(2, fill_to=120), page(3), page(4, fill_to=120)]
    assert failing(run_checks(pages, load_wants(None))) == {"near-empty": [2]}


def test_sparse_page_is_allowed_before_a_part_break():
    part = [body_line(70, size=20, bold=True), body_line(700)]
    pages = [page(1), page(2, fill_to=300), page(3, lines=part), page(4)]
    assert failing(run_checks(pages, load_wants(None))) == {}


def test_sparse_page_without_a_part_break_fails():
    pages = [page(1), page(2, fill_to=300), page(3), page(4)]
    assert failing(run_checks(pages, load_wants(None))) == {"sparse": [2]}


def test_stranded_heading_fails():
    lines = [body_line(70), body_line(690, size=15, bold=True)]
    pages = [page(1), page(2, lines=lines), page(3)]
    assert failing(run_checks(pages, load_wants(None))) == {"heading-at-bottom": [2]}


def test_heading_and_lead_in_followed_by_an_image_passes():
    lines = [body_line(70), body_line(400, size=15, bold=True), body_line(420)]
    p2 = page(2, lines=lines)
    p2.image_bottom = 690.0  # the chart the lead-in introduces
    pages = [page(1), p2, page(3, lines=[body_line(70, mono=True), body_line(700)])]
    assert failing(run_checks(pages, load_wants(None))) == {}
```

```
def test_an_image_counts_toward_page_fill(tmp_path):
    from PIL import Image

    Image.new("RGB", (900, 700), (30, 110, 140)).save(tmp_path / "chart.png")
    md = "## Chart\n\nThe chart below.\n\n![chart](chart.png)\n"
    data, _ = to_pdf(parse(md), "default", base_dir=tmp_path)
    pdf = tmp_path / "chart.pdf"
    pdf.write_bytes(data)
    (shown,) = [p for p in read_pages(pdf) if p.image_bottom]
    assert shown.image_bottom > shown.lines[-1].bottom
    assert shown.content_bottom ⩾ shown.image_bottom


def test_heading_and_lead_in_before_moved_block_fails():
    lines = [body_line(70), body_line(670, size=15, bold=True), body_line(690)]
    nxt = [body_line(70, mono=True), body_line(700)]
    pages = [page(1), page(2, lines=lines), page(3, lines=nxt)]
    assert failing(run_checks(pages, load_wants(None))) == {"heading-at-bottom": [2]}


def test_one_line_section_before_next_heading_passes():
    lines = [body_line(70), body_line(670, size=15, bold=True), body_line(690)]
    nxt = [body_line(70, size=15, bold=True), body_line(700)]
    pages = [page(1), page(2, lines=lines), page(3, lines=nxt)]
    assert failing(run_checks(pages, load_wants(None))) == {}


def test_title_only_document_is_not_a_stranded_heading():
    pages = [page(1, lines=[body_line(70, size=26, bold=True)])]
    assert failing(run_checks(pages, load_wants(None))) == {}


def test_heading_ending_the_last_page_is_not_stranded():
    lines = [body_line(70), body_line(690, size=15, bold=True)]
    pages = [page(1), page(2), page(3, lines=lines)]
    assert failing(run_checks(pages, load_wants(None))) == {}


def test_heading_only_section_after_toc_passes_verify(tmp_path):
    data, _ = to_pdf(parse("---\ntitle: CI\n---\n\n## ok\n"))
    pdf = tmp_path / "ci.pdf"
    pdf.write_bytes(data)
    assert verify(pdf, load_wants(None), tmp_path / "v").ok


def test_title_only_render_passes_verify(tmp_path):
    data, _ = to_pdf(parse("# Hello\n"))
```

```
    pdf = tmp_path / "hello.pdf"
    pdf.write_bytes(data)
    report = verify(pdf, load_wants(None), tmp_path / "v")
    assert report.summary() == "verify: 8/8 checks pass"  # clean: no sheets
    wants = load_wants(None)
    wants["vision"]["sample"] = "per-type"
    report = verify(pdf, wants, tmp_path / "v")
    assert report.ok, report.summary()
    assert "1 sheet for visual review" in report.summary()


def test_bold_code_at_bottom_is_not_a_heading():
    lines = [body_line(70), body_line(690, size=12, bold=True, mono=True)]
    pages = [page(1), page(2, lines=lines), page(3)]
    assert failing(run_checks(pages, load_wants(None))) == {}


def test_text_past_the_margin_fails():
    lines = [body_line(70), body_line(690, x1=600)]
    pages = [page(1), page(2, lines=lines), page(3)]
    assert failing(run_checks(pages, load_wants(None))) == {"overflow": [2]}


def overflow_detail(results):
    return next(r.detail for r in results if r.id == "overflow")


def test_overflow_names_what_runs_over_and_by_how_much():
    right = LETTER[0] - MARGIN_SIDE * PT_PER_IN
    text = page(2, lines=[body_line(70), body_line(690, x1=right + 30)])
    code = page(3, lines=[body_line(70), body_line(690, x1=right + 12, mono=True)])
    table = page(4, lines=[body_line(70), body_line(300, x1=right + 20)])
    table.tables = [(60.0, 290.0, right + 20, 400.0)]
    pages = [page(1), text, code, table, page(5)]
    detail = overflow_detail(run_checks(pages, load_wants(None)))
    assert "p2 text by 30pt" in detail
    assert "p3 code by 12pt" in detail
    assert "p4 table by 20pt" in detail


def wide_table_pdf(tmp_path, ncols: int):
    cols = [f"Column {i}" for i in range(ncols)]
    row = ["m7i.2xlarge", "General purpose", "Up to 12.5 Gbps"]
    row += ["0.4032"] * (ncols - 3)
    md = "## Wide\n\n| " + " | ".join(cols) + " |\n|" + "---|" * ncols + "\n"
    md += ("| " + " | ".join(row) + " |\n") * 6
    data, _ = to_pdf(parse(md), "default")
    pdf = tmp_path / f"wide-{ncols}.pdf"
```

```
    pdf.write_bytes(data)
    return pdf


def test_eleven_column_table_fits_in_smaller_type(tmp_path):
    pages = read_pages(wide_table_pdf(tmp_path, 11))
    assert "overflow" not in failing(run_checks(pages, load_wants(None)))
    sizes = {round(ln.size, 1) for ln in pages[0].lines if ln.text.startswith("m7i")}
    assert sizes == {7.5}


def test_too_wide_a_table_overflows_as_a_table(tmp_path):
    pages = read_pages(wide_table_pdf(tmp_path, 16))
    detail = overflow_detail(run_checks(pages, load_wants(None)))
    assert re.search(r"p1 table by \d+pt", detail), detail


def test_disabled_checks_are_skipped():
    wants = load_wants(None)
    wants["checks"] = ["text-color"]
    pages = [page(1), page(2, fill_to=120), page(3)]
    results = run_checks(pages, wants)
    assert [r.id for r in results] == ["text-color"]


# ---- sampling


def test_sampling_takes_flagged_pages_then_one_per_kind():
    pages = [page(1, kind="cover")] + [page(n, kind="prose") for n in range(2, 9)]
    pages.append(page(9, kind="code"))
    wants = load_wants(None)
    wants["vision"]["sample"] = "per-type"
    pages[3].off_colors = 1  # page 4 flagged
    picked = [p.number for p in sample_pages(pages, run_checks(pages, wants), wants)]
    assert picked[0] == 4  # flagged first
    assert 1 in picked and 9 in picked  # cover and code represented
    assert sum(pages[n - 1].kind == "prose" for n in picked) == 1  # prose covered once


def test_sampling_flagged_takes_only_flagged_pages():
    pages = [page(1, kind="cover")] + [page(n, kind="prose") for n in range(2, 6)]
    wants = load_wants(None)  # default: flagged
    assert sample_pages(pages, run_checks(pages, wants), wants) == []
    pages[2].off_colors = 1  # page 3 flagged
    picked = [p.number for p in sample_pages(pages, run_checks(pages, wants), wants)]
    assert picked == [3]
```

```
def test_sampling_none_returns_nothing():
    wants = load_wants(None)
    wants["vision"]["sample"] = "none"
    assert sample_pages([page(1)], [], wants) == []


# ---- end to end on a real render

DOC = """---
title: Verify me
---

## Section

Some body text that renders black.

| a | b |
|---|---|
| 1 | 2 |
"""


@pytest.fixture
def real_pdf(tmp_path):
    data, _ = to_pdf(parse(DOC))
    path = tmp_path / "doc.pdf"
    path.write_bytes(data)
    return path


def test_real_render_passes_and_writes_sheets(real_pdf, tmp_path):
    wants = load_wants(None)
    wants["vision"]["sample"] = "per-type"
    report = verify(real_pdf, wants, tmp_path / "v")
    assert report.ok, report.summary()
    assert report.sheets and all(
        (tmp_path / "v" / s["path"].split("/")[-1]).exists() for s in report.sheets
    )
    manifest = json.loads((tmp_path / "v" / "verify.json").read_text())
    assert manifest["ok"] is True and manifest["rubric"]


def test_cli_verify_exit_codes(real_pdf, tmp_path, capsys):
    assert main(["verify", str(real_pdf), "--out", str(tmp_path / "a")]) == EXIT_OK
    assert capsys.readouterr().out.startswith("verify: ")

    strict = tmp_path / "wants.yaml"
```

```
    strict.write_text("max_pages: 1\n")  # cover + TOC page break makes 2 pages
    code = main(
        ["verify", str(real_pdf), "--wants", str(strict), "--out", str(tmp_path / "b")]
    )
    assert code == EXIT_CHECKS
    assert "FAIL max-pages" in capsys.readouterr().out

    bad = tmp_path / "bad.yaml"
    bad.write_text("nope: 1\n")
    code = main(["verify", str(real_pdf), "--wants", str(bad)])
    assert code == EXIT_INPUT


def test_example_wants_file_matches_defaults():
    from pathlib import Path

    example = Path(__file__).parent.parent / "wants.example.yaml"
    assert load_wants(str(example)) == DEFAULT_WANTS


def test_reference_sample_gives_one_whole_page_per_kind(tmp_path):
    from importlib import resources

    from rupa.verify import build_references

    sample = (resources.files("rupa") / "references" / "sample.md").read_text()
    data, _ = to_pdf(parse(sample))
    pdf = tmp_path / "sample.pdf"
    pdf.write_bytes(data)
    picked = build_references(pdf, tmp_path / "refs")
    assert set(picked) == {"cover", "prose", "table", "code"}
    assert all((tmp_path / "refs" / f"{k}.png").exists() for k in picked)


def test_references_refuse_a_spilling_sample(tmp_path):
    from rupa.verify import build_references

    code = "\n".join(f"x_{i} = {i}" for i in range(200))
    md = f"## Prose\n\nText.\n\n| a | b |\n|---|---|\n| 1 | 2 |\n\n```python\n{code}\n```\n"
    data, _ = to_pdf(parse(md))
    pdf = tmp_path / "s.pdf"
    pdf.write_bytes(data)
    with pytest.raises(ValueError):
        build_references(pdf, tmp_path / "refs")


def _blank_pdf(path, *sizes):
    import pypdfium2 as pdfium
```

```
    pdf = pdfium.PdfDocument.new()
    for w, h in sizes:
        pdf.new_page(w, h)
    pdf.save(str(path))
    return path


def test_a_pdf_with_no_pages_is_bad_input(tmp_path, capsys):
    # With no pages every check loops over nothing, which used to read 8/8 pass.
    pdf = _blank_pdf(tmp_path / "empty.pdf")
    assert main(["verify", str(pdf)]) == EXIT_INPUT
    assert "no pages" in capsys.readouterr().err


def test_a_zero_size_page_is_bad_input(tmp_path, capsys):
    pdf = _blank_pdf(tmp_path / "zero.pdf", (0, 0))
    assert main(["verify", str(pdf), "--sample", "all"]) == EXIT_INPUT
    assert "no size" in capsys.readouterr().err


def test_sheet_size_is_capped_on_a_huge_page(tmp_path):
    from rupa.verify import MAX_SHEET_PX, render_page_png

    pdf = _blank_pdf(tmp_path / "big.pdf", (2000, 2000))  # ~28 in square
    img = render_page_png(pdf, 1, 300)  # uncapped this is 8333 px a side
    assert max(img.size) ⩽ MAX_SHEET_PX
```

Checkpoint: `uv run pytest -q` shows 40 passed, and `rupa verify hello.pdf` prints a line starting with `verify:` and writes a `hello.verify/` folder with `verify.json` (plus a sheet for any flagged page).

Checkpoint in Claude Code: with the plugin loaded, ask for a one-page PDF and to verify it. Claude should render, run `rupa verify`, open the sheet, give a verdict per sheet, and fix the Markdown if something fails.

Step 9: Measure

`bench/bench.sh` sets up the four arms and runs them. The `awk` line builds the render-only plugin by dropping everything from `## Verify` up to the skill's closing line, and the `sed` line builds the per-type plugin by adding `--sample per-type` to the verify command:

```
#!/usr/bin/env bash
# Compare Claude Code cost on the same task across four arms:
#   baseline       no plugin: Claude writes the HTML/CSS and finds its own PDF tool
#   rupa           plugin, render only (the skill's Verify section is removed)
#   rupa-pertype   plugin, verify with a sheet per page type as well as flagged pages
#   rupa-verify    plugin as shipped: render, then verify; sheets only for flagged pages
#
# Usage: bench/bench.sh [runs]        default 1 run per arm (a pilot)
#        MODEL=claude-sonnet-5-5 bench/bench.sh 3
#        TASK=guide bench/bench.sh 3    prompt from bench/tasks/<TASK>.txt
#                                         (default: report, a 2-page prose report)
#        ARMS="rupa rupa-verify" bench/bench.sh 3   only these arms (default: all four)
# Each run starts in a fresh empty directory, so no arm can see this repo.
set -euo pipefail

RUNS="${1:-1}"
ROOT="$(cd "$(dirname "$0")/.." && pwd)"
TASK_NAME="${TASK:-report}"
TASK_FILE="$ROOT/bench/tasks/$TASK_NAME.txt"
[[ -f "$TASK_FILE" ]] || { echo "bench: no task $TASK_FILE" >&2; exit 2; }
TASK_PROMPT="$(cat "$TASK_FILE")"
RESULTS="$ROOT/bench/results/$(date +%Y%m%d-%H%M%S)-$TASK_NAME"
TOOLS="Bash Write Read Edit Skill"
MODEL_FLAG=()
[[ -n "${MODEL:-}" ]] && MODEL_FLAG=(--model "$MODEL")

mkdir -p "$RESULTS"
cp "$TASK_FILE" "$RESULTS/task.txt"
command -v rupa >/dev/null || { echo "bench: rupa is not on PATH" >&2; exit 2; }
export RUPA_LOG=off  # keep benchmark runs out of the real-use failure log

# Variant arms use a copy of the plugin with an edited skill.
# Copy only what loads, not evals/ and its results.
plugin_copy() {  # $1 = directory to create
  mkdir -p "$1"
  cp -r "$ROOT/plugin/.claude-plugin" "$ROOT/plugin/hooks" "$ROOT/plugin/skills" "$1/"
}
SKILL="$ROOT/plugin/skills/render/SKILL.md"

# Render only: the skill stops before "## Verify".
```

```
NOVERIFY="$RESULTS/plugin-noverify"
plugin_copy "$NOVERIFY"
awk '/^## Verify/ {skip=1} /^Do not add styling/ {skip=0} !skip' \
  "$SKILL" > "$NOVERIFY/skills/render/SKILL.md"

# Per-type: verify also makes one sheet per page type, even on a clean PDF.
PERTYPE="$RESULTS/plugin-pertype"
plugin_copy "$PERTYPE"
sed 's/`rupa verify out\/report.pdf`/`rupa verify --sample per-type out\/report.pdf`/' \
  "$SKILL" > "$PERTYPE/skills/render/SKILL.md"
grep -q -- '--sample per-type' "$PERTYPE/skills/render/SKILL.md" \
  || { echo "bench: could not make the per-type skill; SKILL.md changed?" >&2; exit 2; }

ARMS="${ARMS:-baseline rupa rupa-pertype rupa-verify}"

run_arm() {  # $1 = arm name, remaining args = extra claude flags
  local arm="$1"; shift
  [[ " $ARMS " == *" $arm "* ]] || return 0
  for i in $(seq 1 "$RUNS"); do
    local work; work="$(mktemp -d)"
    echo "bench: $arm run $i" >&2
    ( cd "$work" && claude -p "$TASK_PROMPT" --output-format json --allowedTools "$TOOLS" \
        ${MODEL_FLAG[@]+"${MODEL_FLAG[@]}"} "$@" ) > "$RESULTS/$arm-$i.json" 2> "$RESULTS/
$arm-$i.err" || true
    if [[ -s "$work/report.pdf" ]]; then
      cp "$work/report.pdf" "$RESULTS/$arm-$i.pdf"
    fi
    rm -rf "$work"
  done
}

run_arm baseline
run_arm rupa --plugin-dir "$NOVERIFY"
run_arm rupa-pertype --plugin-dir "$PERTYPE"
run_arm rupa-verify --plugin-dir "$ROOT/plugin"

python3 "$ROOT/bench/summarize.py" "$RESULTS"
```

`bench/summarize.py` reads one results folder and prints the tables:

```
"""Summarize a bench results directory: one row per run, then medians per arm.

Turns and cache reads come first because they drive cost: every turn re-reads
the whole context from cache, while output tokens are a small share.
"""
```

```
import json
import statistics
import sys
from pathlib import Path

# rupa-flagged is the old name of the flagged arm, kept so older results read.
ARMS = ("baseline", "rupa", "rupa-flagged", "rupa-pertype", "rupa-verify")
COLS = ("turns", "cache_read", "cache_write", "input", "output", "cost_usd", "secs")


def row(path: Path) -> dict | None:
    try:
        r = json.loads(path.read_text())
    except (OSError, json.JSONDecodeError):
        return None
    u = r.get("usage", {})
    return {
        "turns": r.get("num_turns", 0),
        "cache_read": u.get("cache_read_input_tokens", 0),
        "cache_write": u.get("cache_creation_input_tokens", 0),
        "input": u.get("input_tokens", 0),
        "output": u.get("output_tokens", 0),
        "cost_usd": r.get("total_cost_usd", 0.0),
        "secs": round(r.get("duration_ms", 0) / 1000),
        "pdf": path.with_suffix(".pdf").exists(),
    }


def fmt(v) -> str:
    if isinstance(v, float):
        return f"{v:.3f}"
    if isinstance(v, bool):
        return "yes" if v else "NO"
    return f"{v:,}"


def main(results: Path) -> None:
    runs = {arm: [] for arm in ARMS}
    for f in sorted(results.glob("*.json")):
        arm = f.stem.rsplit("-", 1)[0]
        if arm in runs and (r := row(f)):
            runs[arm].append(r)
    arms = [a for a in ARMS if runs[a]]

    header = ("arm", "run", *COLS, "pdf")
    print("\t".join(header))
    for arm in arms:
        for i, r in enumerate(runs[arm], 1):
```

```
            print("\t".join([arm, str(i), *(fmt(r[c]) for c in COLS), fmt(r["pdf"])]))

    print("\nmedians (runs without a PDF excluded)")
    print("\t".join(("arm", "n", *COLS)))
    base = None
    for arm in arms:
        ok = [r for r in runs[arm] if r["pdf"]]
        if not ok:
            print(f"{arm}\t0\t(no successful runs)")
            continue
        med = {c: statistics.median(r[c] for r in ok) for c in COLS}
        med = {c: v if c == "cost_usd" else round(v) for c, v in med.items()}
        base = base or (med if arm == "baseline" else None)
        print("\t".join([arm, str(len(ok)), *(fmt(med[c]) for c in COLS)]))
    if base:
        print("\nvs baseline (median turns, median cost)")
        for arm in arms[1:]:
            ok = [r for r in runs[arm] if r["pdf"]]
            if ok and base["turns"] and base["cost_usd"]:
                t = statistics.median(r["turns"] for r in ok) / base["turns"] - 1
                c = statistics.median(r["cost_usd"] for r in ok) / base["cost_usd"] - 1
                print(f"{arm}\tturns {t:+.0%}\tcost {c:+.0%}")


if __name__ == "__main__":
    main(Path(sys.argv[1]))
```

Before spending anything, test the plumbing with a stand-in for `claude` that writes a fake `report.pdf` and prints canned JSON. During the build this showed each arm getting the right plugin and the summary computing the comparisons correctly. `bench/fake/claude`:

```
#!/usr/bin/env bash
# Stand-in for `claude` to test bench.sh plumbing without spending tokens.
# Usage: PATH="$PWD/bench/fake:$PATH" bench/bench.sh 2
# Reports fewer turns for the plugin arms so the comparison lines have
# something to show. The numbers are made up; only the plumbing is real.
plugin=""
while [[ $# -gt 0 ]]; do [[ "$1" == "--plugin-dir" ]] && plugin="$2"; shift; done
turns=20
if [[ -n "$plugin" ]]; then
  turns=12
  grep -q "## Verify" "$plugin/skills/render/SKILL.md" && turns=15
  grep -q -- "--sample per-type" "$plugin/skills/render/SKILL.md" && turns=17
fi
printf '%%PDF-1.4 fake\n' > report.pdf
cat <<JSON
```

```
{"num_turns": $turns, "total_cost_usd": 0.$turns, "duration_ms": 30000,
 "usage": {"input_tokens": 5, "cache_creation_input_tokens": 50000,
           "cache_read_input_tokens": $((turns * 40000)), "output_tokens": 4000}}
JSON
```

```
PATH="$PWD/bench/fake:$PATH" bench/bench.sh 2
# ...
# rupa          turns -40%   cost -40%
# rupa-pertype  turns -15%   cost -15%
# rupa-verify   turns -25%   cost -25%
```

Those numbers come from the stand-in, not from Claude.

Then run the pilot, one run per arm, and check the transcripts:

```
chmod +x bench/bench.sh
bench/bench.sh
```

Prompts live in `bench/tasks/`, one file per task, and `TASK=<name>` picks one (default `report`, the two-page prose report). `guide` is a harder three-page task with a five-column table, a CLI example and a 25-line JSON policy, closer to where hand-written HTML has to work for its layout. `tight` is the guide squeezed into two pages: the required parts alone fit only at tight type, so any real prose goes over the budget and the run has to cut. It exists to test how Rupa runs cut, so run it on the Rupa arms only: `ARMS="rupa rupa-verify"` picks which arms run (default all four). `wide` is a three-page EC2 sizing guide whose table has eleven columns, every one required; at the default type eight columns fit and ten already run off the page, so a run has to split the table rather than drop columns. The render-only `rupa` arm never runs verify, so it is the one most likely to ship a clipped table. Each results folder is named after its task and keeps a copy of the prompt in `task.txt`.

If all four arms behaved, run three per arm on a pinned model:

```
MODEL=claude-sonnet-5-5 bench/bench.sh 3
```

Record the medians in your README with the date, model and task. If a Rupa arm isn't cheaper, read its transcript before changing anything else.

Step 10: What to build next

In rough order of value:

1. **Re-run** `TASK=guide` to confirm the height-based cut estimate brings the cut rounds down to one. Then a harder benchmark task (tables, code, a wide table that must be split), where hand-written HTML should struggle more than it did on a two-page prose report.
2. **A second theme**, such as your dark slate and cyan look for screen HTML, with its own reference pages; the golden-render test picks it up from `THEMES` with no new test code.
3. **Callouts** (note, warning) through a small markdown-it plugin, so Claude never needs raw HTML for them.
4. **A** `--watch` **mode** for live preview while you edit by hand.
5. **More targets** (DOCX through python-docx) behind the same contract and exit codes.
6. **Harness integration.** Rupa is a plain CLI, so the Pramā harness can call it the same way Claude Code does.

Turning the build into a showcase

If this becomes a second project about how you built Rupa, capture material as you go rather than reconstructing it afterward.

- **One commit per milestone,** with the milestone number in the message. The git log becomes the outline.
- **A decision log.** The Key decisions table in Part 1 is the start; add a row whenever you change your mind, with the reason.
- **The numbers.** The three-arm tables, with model, date and task. Lead with turns: that's the finding.
- **Before and after artifacts.** The same report produced by hand-written HTML and by Rupa, side by side.
- **The failures.** The WeasyPrint `BaseException` trap, the API change, the background rectangle, and the code wrap that only the visual review caught are exactly the kind of detail that makes a build write-up credible.
- **The verification records.** Each `verify.json` and its sheets show what was checked and what was found. The appendix of this document is one.

A natural shape for a write-up or Loom entry: the problem (each turn re-reads everything, and markup work adds turns), the split (content versus form), the contract, the verification loop, the measurement, and what the numbers showed. The showcase can itself be rendered by Rupa.

Appendix: Verification record

This is the record of verifying this document with the loop it describes. Each pass ran `rupa verify` with the default wants, then a visual review of the sheets against the rubric. The verdicts are Claude's, written one line per sheet as the skill asks.

Pass 1: the first delivered version

The first version ran to 31 pages, before the verification sections existed.

Code checks: `verify: 7/8 checks pass (FAIL near-empty p11,30); 5 sheets`

Sheet	Verdict
p1 cover	PASS
p7 table	PASS
p11 prose	FAIL page balance: one table row (Sustainability) alone on a page before Part 3
p21 code	FAIL overflow: CSS lines broken mid-word, `1px s` / `olid`
p30 prose	FAIL page balance: the last four items of a list alone on a page before the closing part

Two of the three failures were also flagged by the code checks. The third, the mid-word break, was visible only in the image: the text stayed inside the margins, so `overflow` passed.

Fixes. The lone row was a content fix: Cost optimization and Sustainability became one row. The list tail resolved once the new sections moved the page breaks. The mid-word break could only be fixed in the theme, so the loop would have reported it rather than patched it. It was fixed by hand in `print.css` (plain `pre-wrap`), and the long lines in the source files were shortened.

Pass 2: after adding verification

Code checks: `verify: 8/8 checks pass; 4 sheets`

All four rendered pages passed. The review failed the reference instead: the code reference showed the top of a code block that continued onto a sixth page, so the comparison was against half a page. An edit to the sample document had added blank lines. The sample was trimmed, and `rupa refs` now refuses a sample whose page for any type spills over. Two tests cover that.

Final pass

Code checks: `verify: 8/8 checks pass; 4 sheets for visual review in verify/`

Sheet	Verdict
p1 cover	PASS
p10 table	PASS
p12 prose	PASS
p39 code	PASS

Cost of this pass: 4 sheets at about 2,000 input tokens each, against roughly 176,000 tokens to look at all 64 pages at full resolution.